

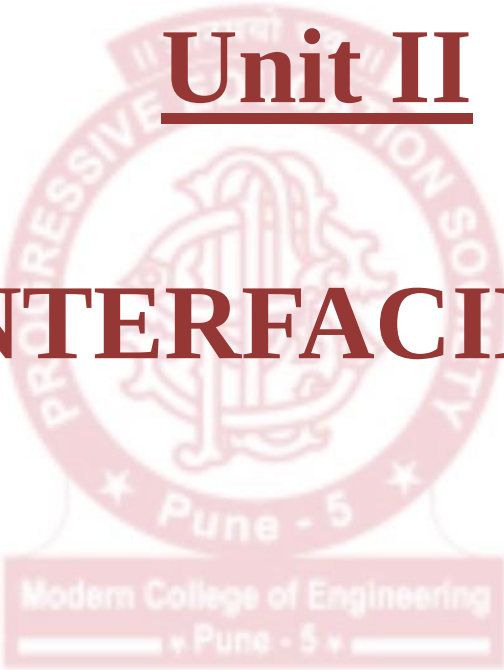


Progressive Education Society's
Modern College of Engineering, Pune - 411005
Department of Electronics & Telecommunication

AY-2024-25 Semester - I

Microcontroller and Applications

Course In-charge
Mrs. R. S. Chaudhari



Unit II

INTERFACING-I

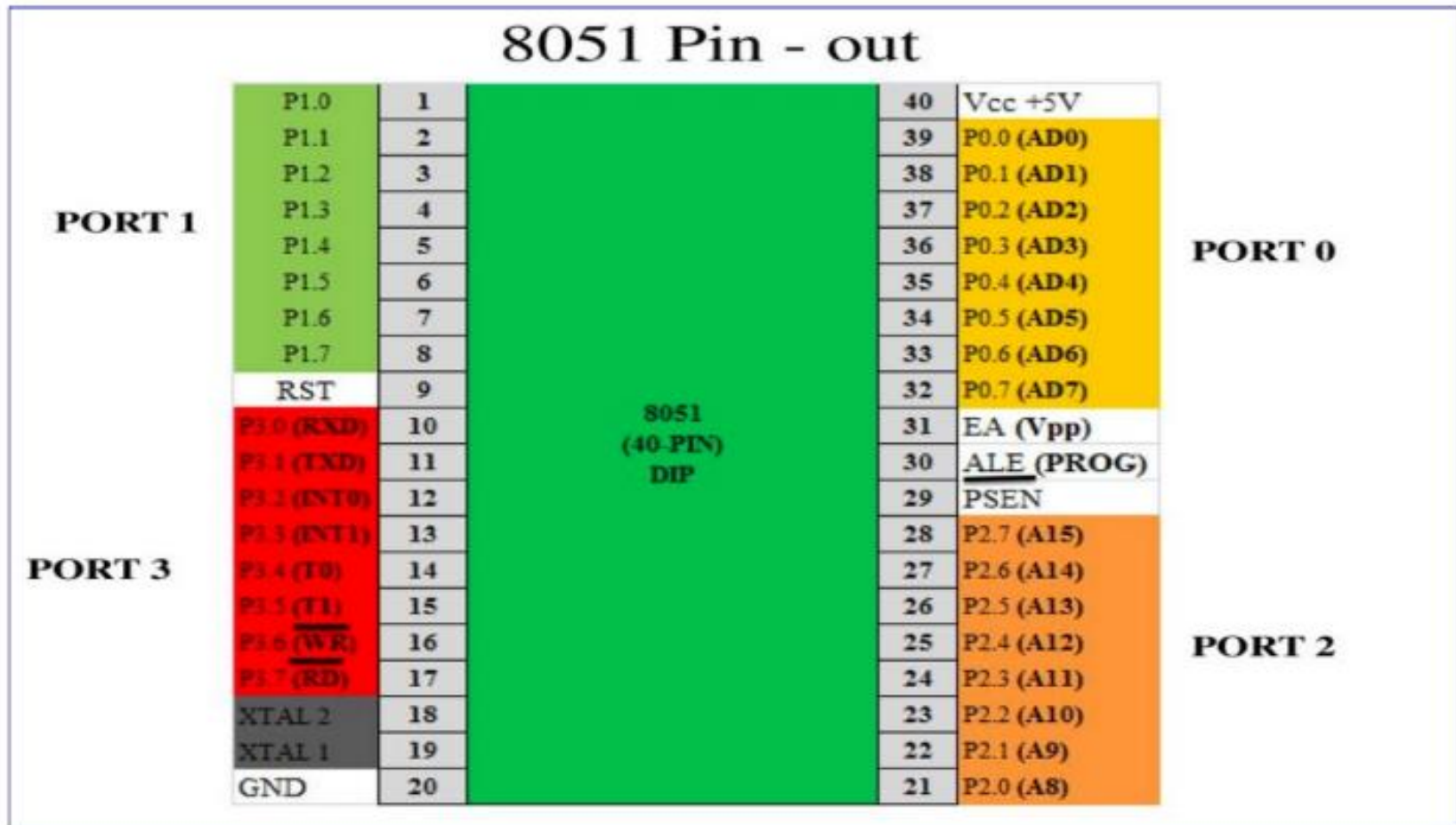
Syllabus

- Software and Hardware tools for development of microcontroller based systems such as -
- Assemblers, compilers, IDE, Emulators, debuggers, programmers, development board
- DSO, Logic Analyzer
- Interfacing LED with and without interrupt
- Keypads
- Seven Segment multiplexed Display
- LCD
- ADC Interfacing
- All Programs in embedded C language.

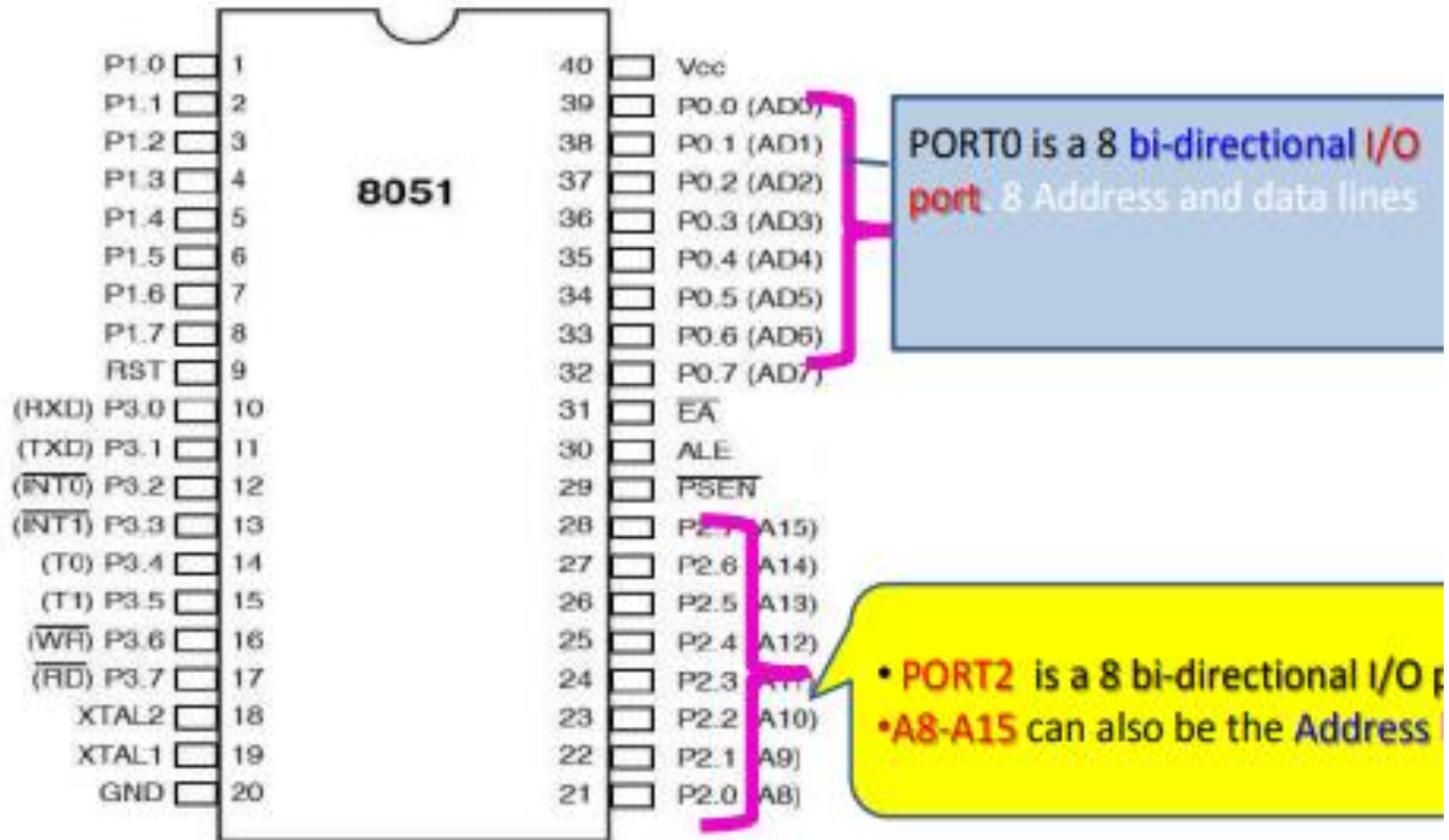
Pin diagram and its functioning



8051 Pin diagram and its functioning



8051 Ports

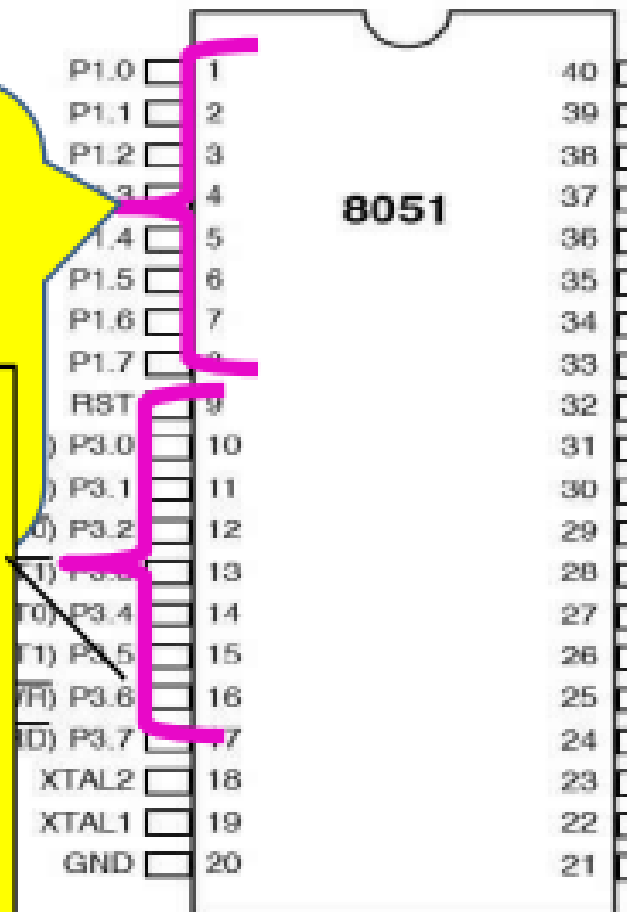


8051 Ports

- **PORT 1** is a bi-directional I/O port.

- **P1.0**
- **P1.1**
- **P1.2**
- **P1.3**

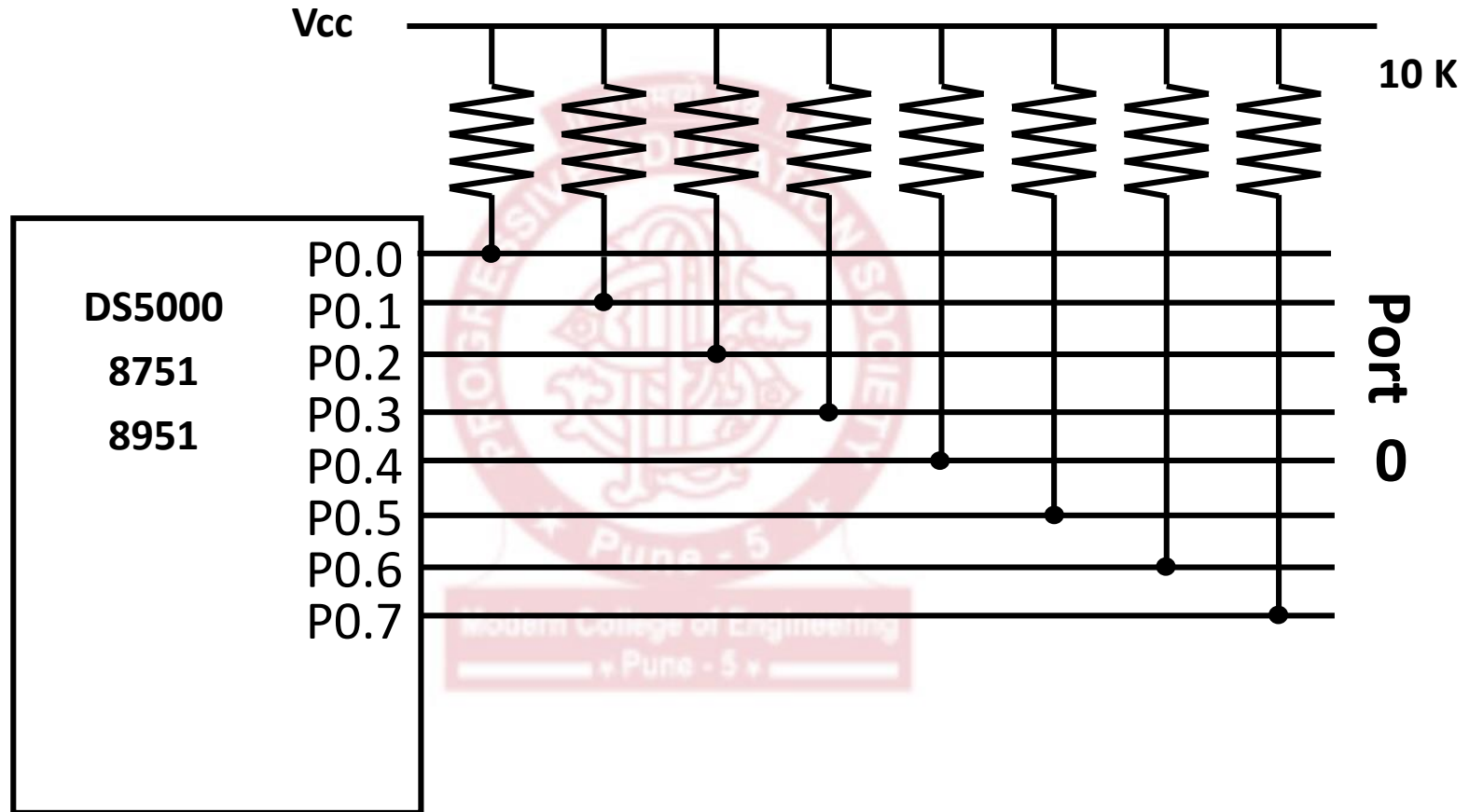
P3.0 / RXD can also be the Receive data
P3.1/TXD can also be used Transmit
P3.2 / INT0 can also be used for external INT0.
P3.3 / INT1 can also be used for external INT0.
P3.4 / T0 can be used as Inputs to timer 0
P3.5 / T1 can be used as Inputs to timer 1
P3.6 / WR can be used as write data to ext memory
P3.7/ RD can be used as reading data from ext memory



Important Pins (IO Ports)

- **One of the most useful features of the 8051 is that it contains four I/O ports (P0 - P3)**
- Port 0 (pins 32-39) : P0 (P0.0~P0.7)
 - 8-bit R/W - General Purpose I/O
 - Or acts as a multiplexed low byte address and data bus for external memory design
- Port 1 (pins 1-8) : P1 (P1.0~P1.7)
 - Only 8-bit R/W - General Purpose I/O
- Port 2 (pins 21-28) : P2 (P2.0~P2.7)
 - 8-bit R/W - General Purpose I/O
 - Or high byte of the address bus for external memory design
- Port 3 (pins 10-17) : P3 (P3.0~P3.7)
 - General Purpose I/O
 - if not using any of the internal peripherals (timers) or external interrupts.
- **Each port can be used as input or output (bi-direction)**

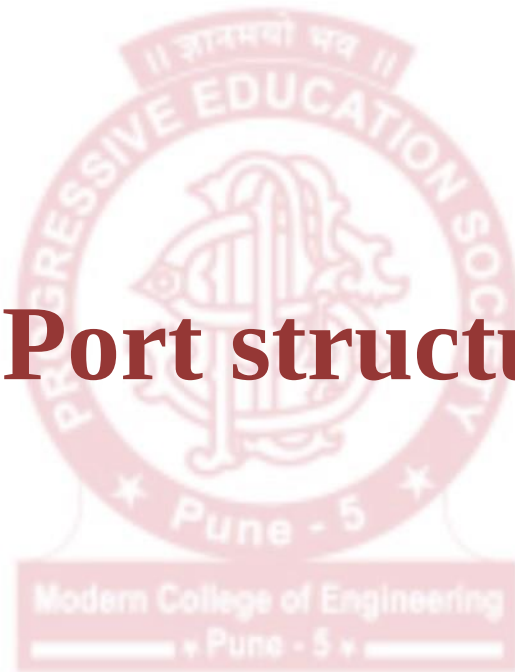
8051 Port 0 with pull-up resistors



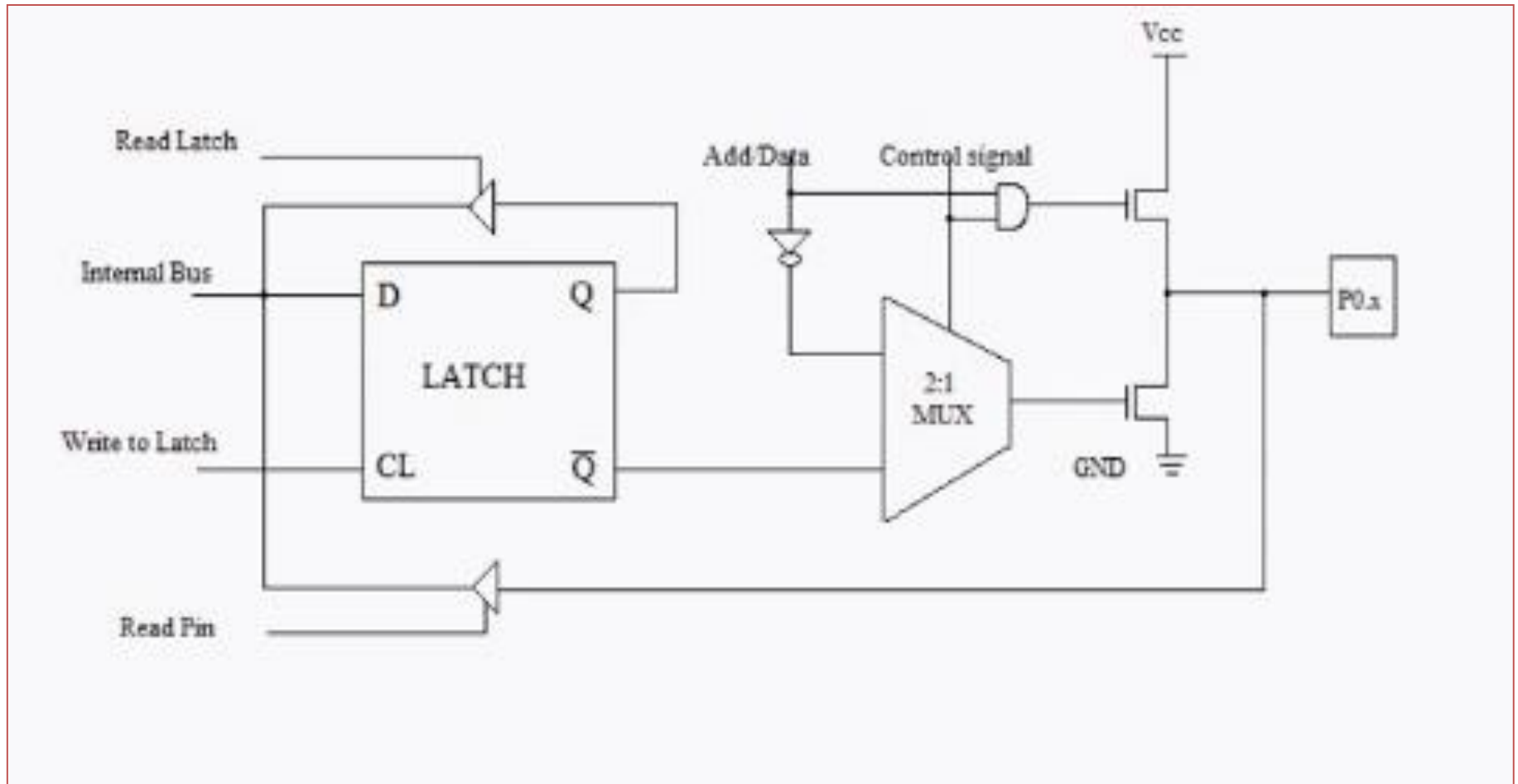
8051 Port 0 with pull-up resistors

- The reason for not having pull-ups for port 0 internally is because this port has multiplexing of address and data lines. Hence it depends on what purpose we are going to use this port.
- When we use external pull-ups it becomes general purpose port.
- Again, port 0 is open drain hence it requires pull-ups and all other ports are not having open drain its not left to user.
- If port 0 has to be used as i/o port its must to connect pull up resistor.

Port structure

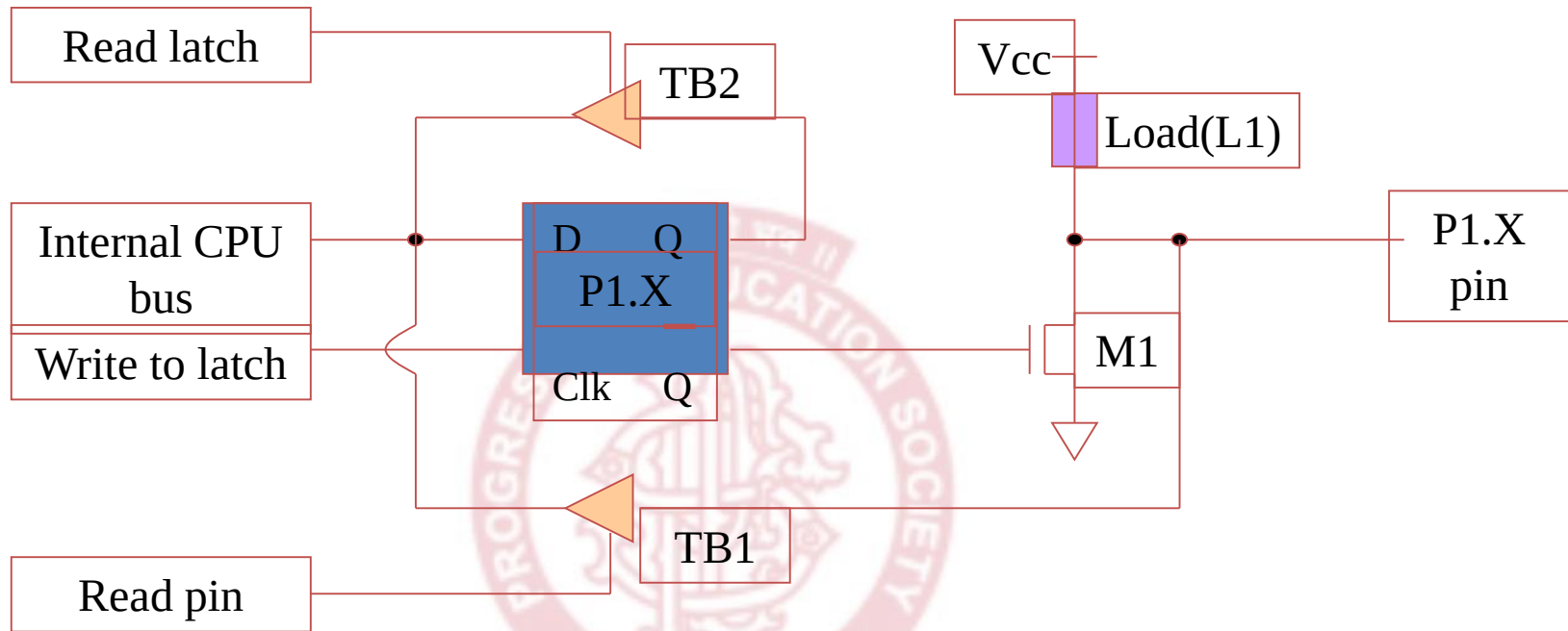


Hardware structure of Port 0



For port 0 to make as input port control signal need to be 1

Hardware Structure of I/O Pin



Each pin of I/O ports

Internally connected to CPU bus

A **D latch** store the value of this pin

Write to latch=1 : write data into the D latch

2 **Tri-state** buffer :

TB1: controlled by “Read pin”

Read pin=1 : really read the data present at the pin

TB2: controlled by “Read latch”

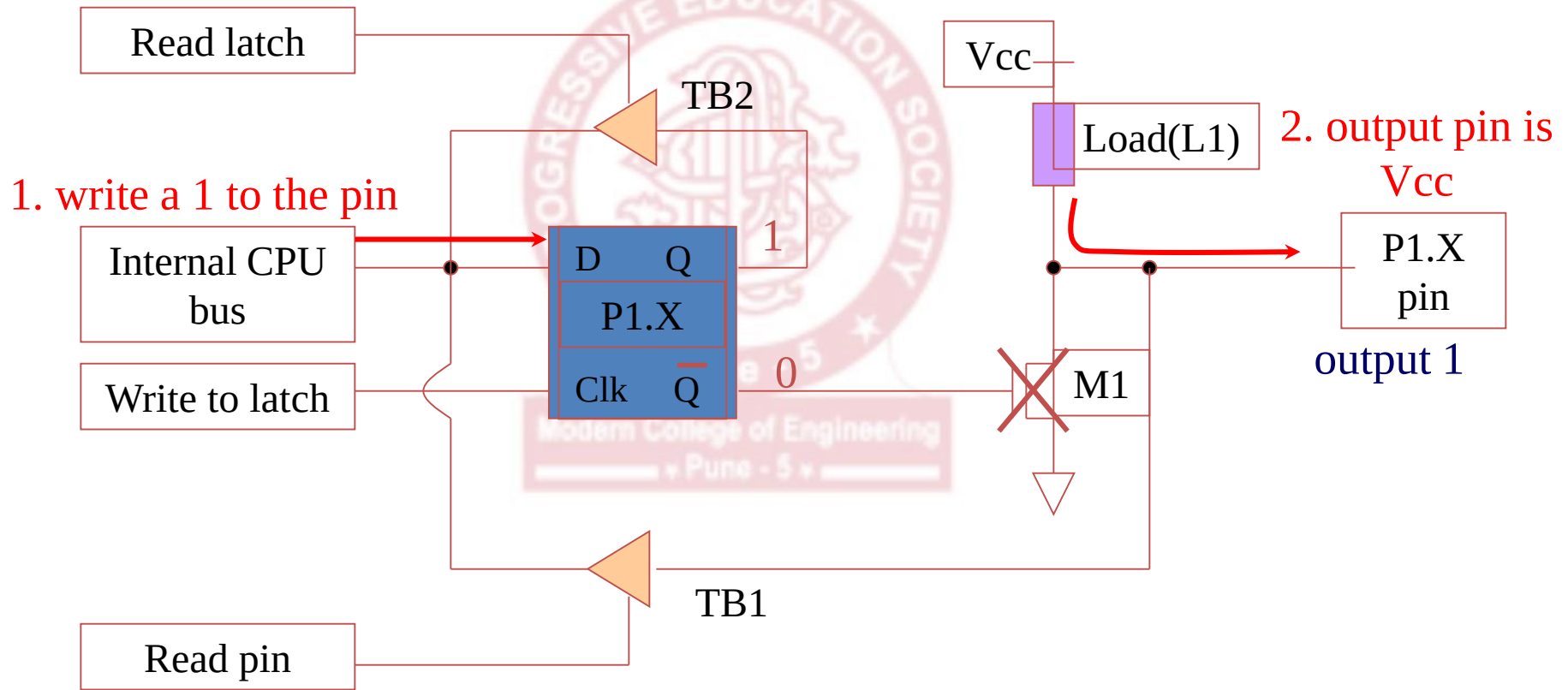
Read latch=1 : read value from internal latch

A **transistor** M1 gate

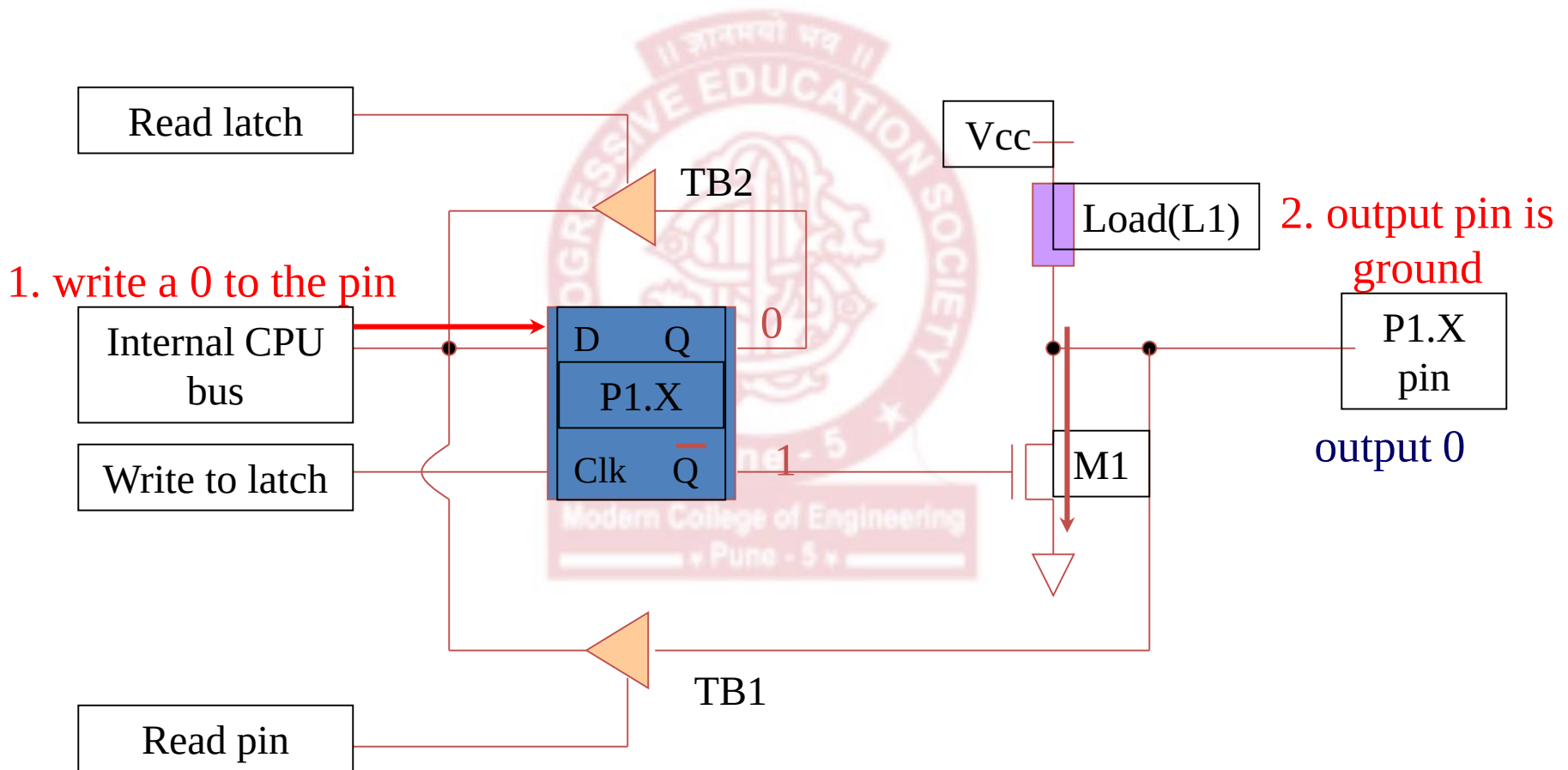
Gate=0: open

Gate=1: close

Writing “1” to Output Pin P1.X port acting as input port

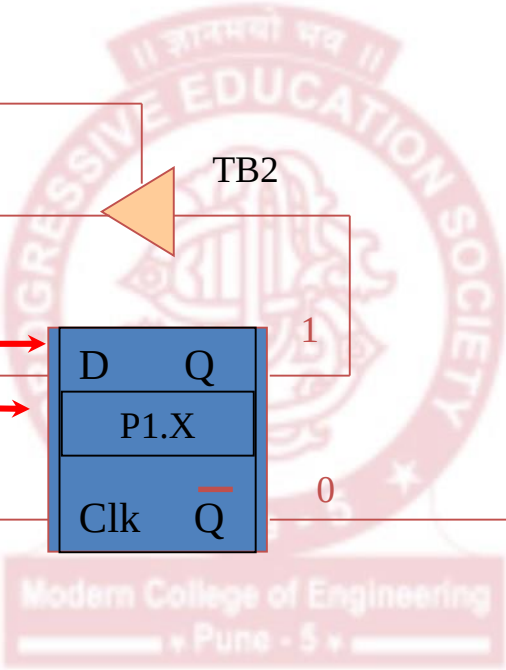


Writing “0” to Output Pin P1.X port acting as output port

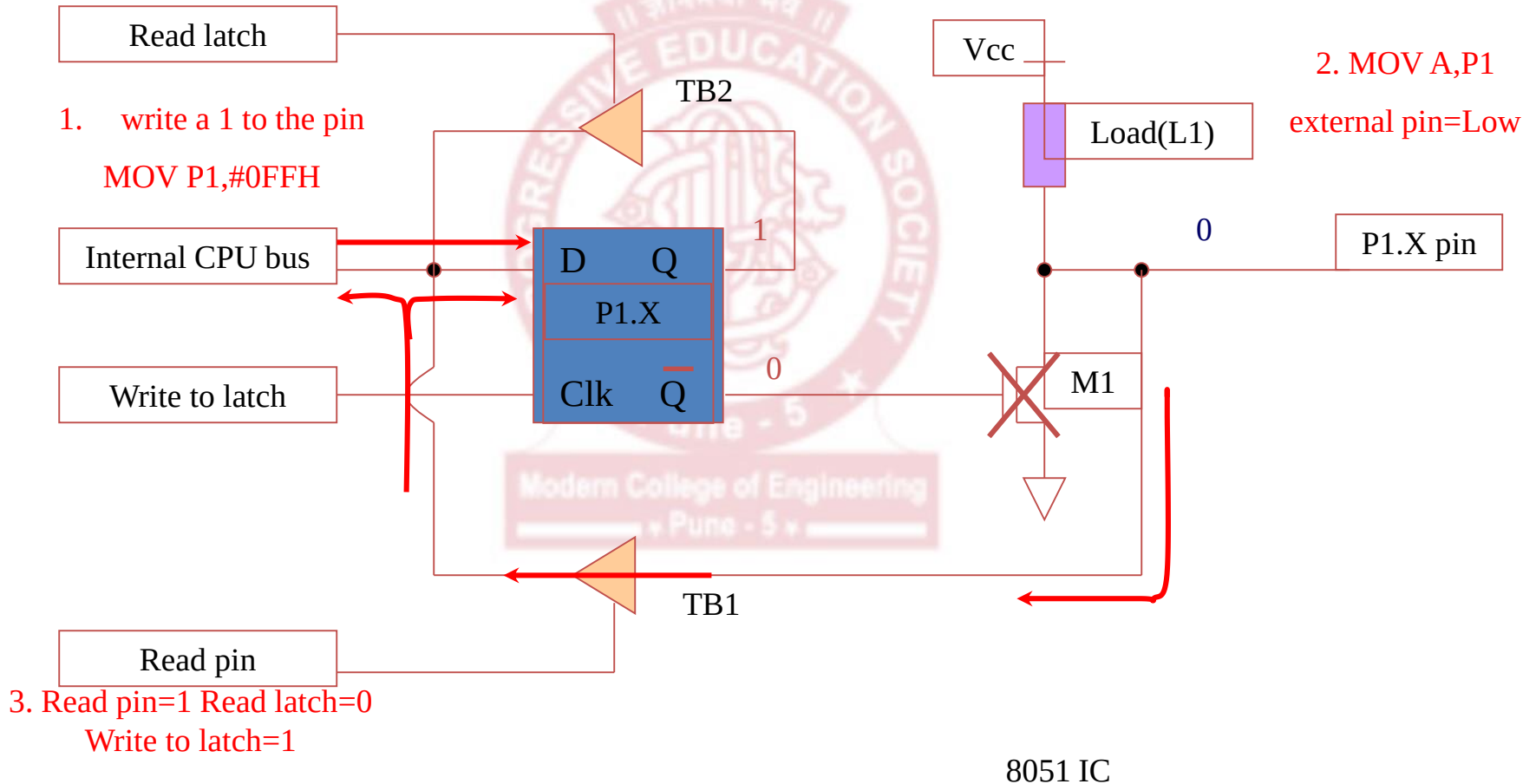


The diagram shows a D flip-flop with the following connections:

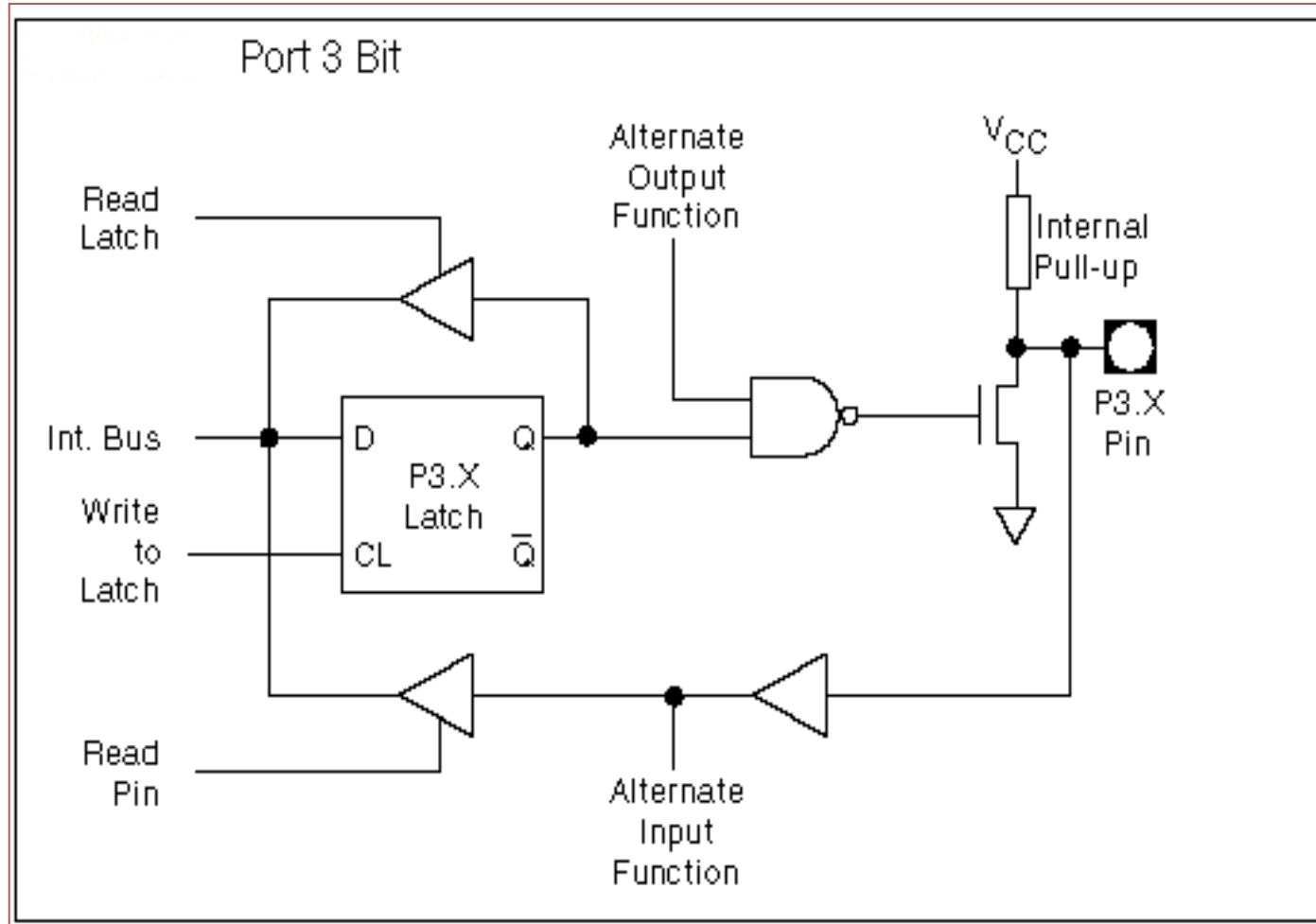
- Input D:** Connected to the output Q via a buffer labeled TB2.
- Input Q:** Connected to a logic 1 source.
- Input Clk:** Connected to a logic 0 source.
- Output:** Labeled P1.X.



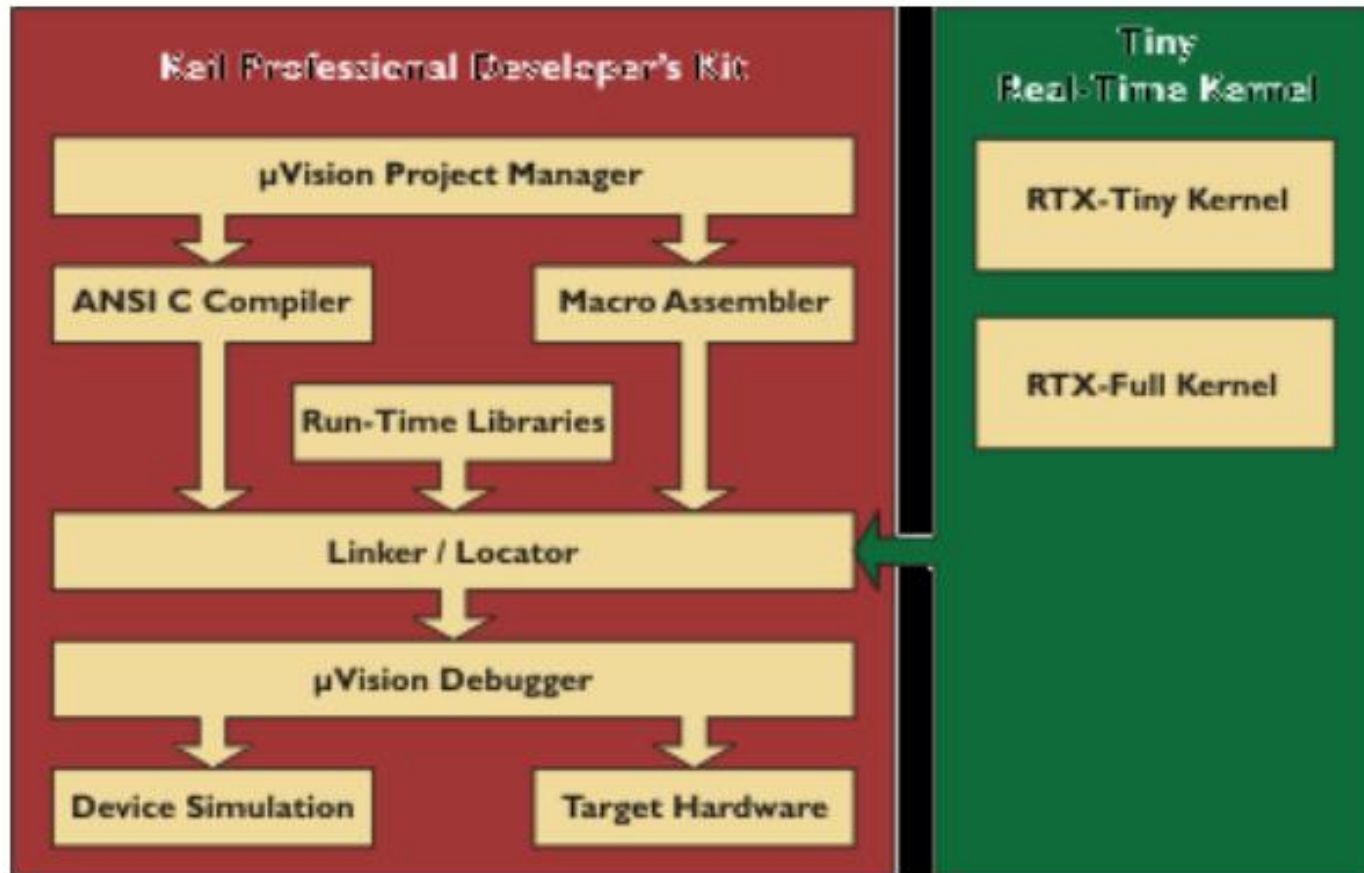
Reading “Low” at Input Pin



8051 Port 3 Bit Latches and I/O Buffers



IDE



Assembler and Compiler

A51 Assembler

- It is a macro assembler for the 8051 family of microcontrollers.
- It supports all 8051 derivatives.
- It translates symbolic assembly language mnemonics into object code where the utmost speed, small code size, and hardware control are critical.
- The macro facility speeds development and conserves maintenance time.
- The A51 assembler supports symbolic access to all features of the 8051 architecture.
- .ASM to .OBJ

C51 C Compiler

- The C51 Compiler allows you to write 8051 microcontroller applications in C that, once compiled, have the efficiency and speed of assembly language.
- Language extensions in the C51 Compiler give you full access to all resources of the 8051.
- The C51 Compiler translates C source files into relocatable object modules which contain full symbolic information.
- In addition to the object file, the compiler generates a listing file which may optionally include symbol table and cross reference information.
- .C to .OBJ

When the .c file is compiled, .lst and .obj files are generated by the assembler

Linker and Simulator

Linker/Locator

- Linker/locator combines object modules and creates executable 8051 programs.
- The linker resolves external and public references and assigns absolute or fixed addresses to relocatable program segments.
- Linker processes object files created by the Keil C51 Compiler and A51 Assembler
- It outputs an absolute object module that may be loaded practically into any emulator, the Keil μ Vision Debugger, or the Object-HEX converter (to create an Intel HEX file).

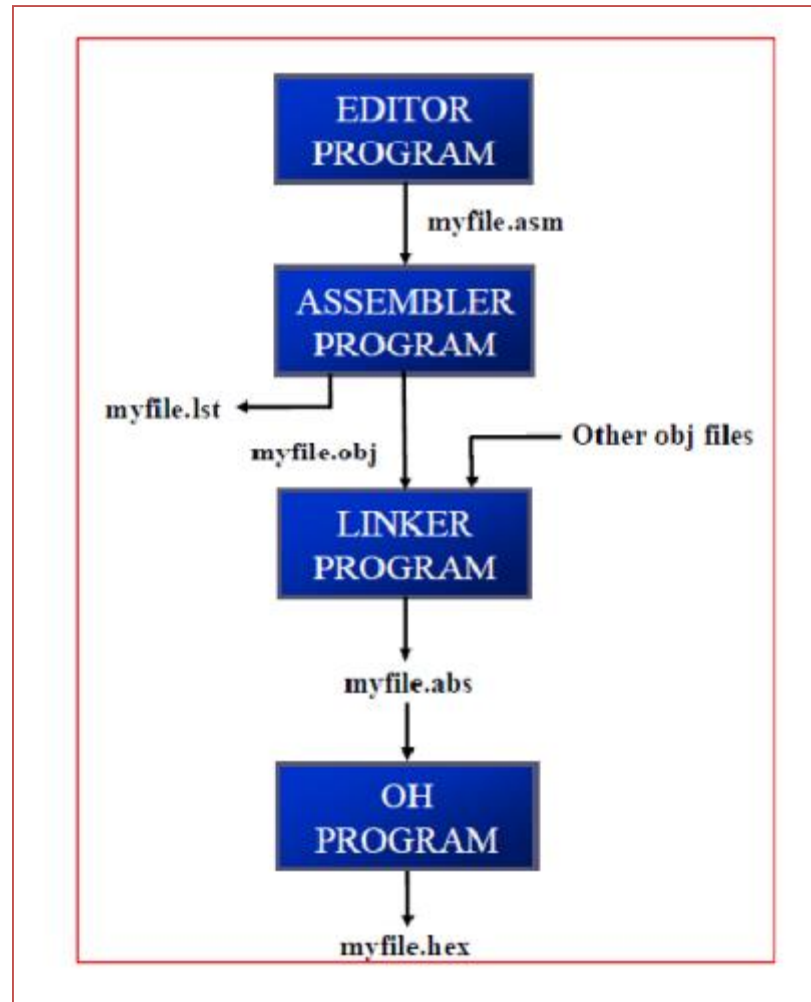
Simulator

- The simulator takes less time debugging simple program errors.
- It lets them learn about things like on-chip peripherals and addressing modes without designing real hardware.
- It requires less technical support and are able to get up-to-speed with the tools faster.
- It makes it easy to write and test code and learn about programming your microcontroller.
- It is easier to explain complex problems with help of a simulator.
- It is easier to discover if a problem is in the hardware or software when you use a simulator. For example, if the application works in the simulator and if it works in the emulator, there's most likely a problem with the target hardware.
- It requires no setup time. An emulator may require configuration and a target board before you can debug.

Steps of assembly language program

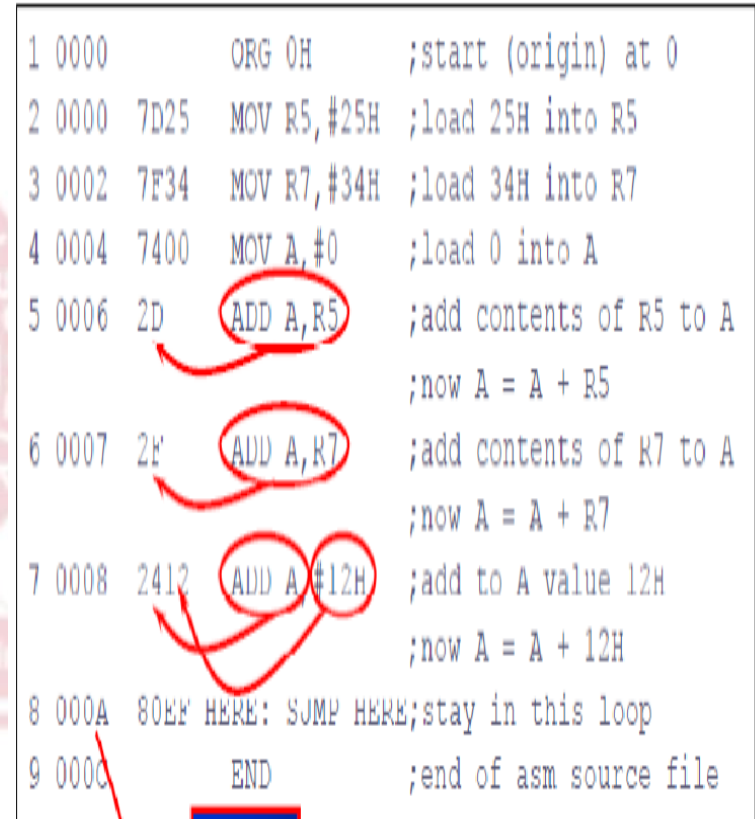
1. First use an editor to type a program, Using editors or word processors to create and/or edit the program. The editor produces an ASCII file which can be saved with the extension .asm
2. The “C” source file containing the program code created in step 1 is fed to an 8051 assembler
3. The assembler converts the instructions into machine code
4. The assembler will produce an object file and a list file
5. The extension for the object file is “obj” while the extension for the list file is “lst”
6. Assembler require a third step called linking
7. The linker program takes one or more object code files and produce an absolute object file with the extension “abs”
8. This abs file is used by 8051 trainers that have a monitor program
9. Next the “abs” file is fed into a program called “OH” (object to hex converter) which creates a file with extension “hex” that is ready to burn into ROM
10. This program comes with all 8051 assemblers
11. Recent Windows-based assemblers combine step 2 through 4 into one step

IDE



List file

- The .lst (list) file, which is optional, is very useful to the programmer
- It lists all the opcodes and addresses as well as errors that the assembler detected
- The programmer uses the lst file to find the syntax errors or debug
- The program counter points to the address of the next instruction to be executed
- As the CPU fetches the opcode from the program ROM, the program counter is increasing to point to the next instruction
- The program counter (PC) is 16 bits wide
- This means that it can access program addresses 0000 to FFFFH, a total of 64K bytes of code



The screenshot shows an assembly list file with 9 lines of code. Red circles highlight the instruction mnemonics 'ADD A,R5', 'ADD A,R7', and 'ADD A,#12H'. Red arrows indicate the sequence of execution from line 5 to line 6, and then from line 6 to line 7. A red line is drawn under line 9, and a blue rectangle is placed below it.

```
1 0000      ORG 0H      ;start (origin) at 0
2 0000 7D25  MOV R5,#25H ;load 25H into R5
3 0002 7F34  MOV R7,#34H ;load 34H into R7
4 0004 7400  MOV A,#0    ;load 0 into A
5 0006 2D     ADD A,R5    ;add contents of R5 to A
                        ;now A = A + R5
6 0007 2F     ADD A,R7    ;add contents of R7 to A
                        ;now A = A + R7
7 0008 2412   ADD A,#12H  ;add to A value 12H
                        ;now A = A + 12H
8 000A 80EF  HERE: SJMP HERE;stay in this loop
9 000C      END          ;end of asm source file
```


DSO and Logic analyzer

- Both are essential tools in electronics and digital circuit debugging
- DSO is used to observe the changing signal voltages in a circuit over time. Particularly for analyzing analog waveforms and mixed-signal systems.
- Its main application is debugging and troubleshooting analog circuits.
- DSO can display analog as well as digital signals very well. It can also store them for later use.
- Choosing the right tool for the job can be a crucial step when debugging circuits.
- At its core, an oscilloscope is useful for measuring and visualizing analog signals with 1-4 channels.
- An oscilloscope can help you visualize analog waveforms and measure various characteristics, such as amplitude, ringing, transient signals, phase, and unwanted pulses. Even when debugging digital systems, you can easily measure signal integrity with an oscilloscope.
- A logic analyzer is best for digital systems with more than 4 channels, nearly 34 to 136 channels.
- However, if you need to monitor and debug the output of a digital system or capture digital communication, such as serial and I2C, a logic analyzer is often the better tool. Logic analyzers generally offer more channels than an oscilloscope and can display data in state mode, where one of the channels acts as a clock.

DSO and Logic analyzer

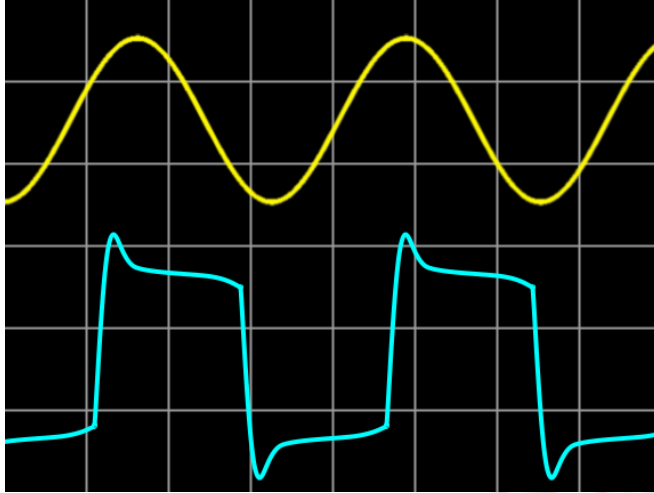
Oscilloscope

- Measure and display analog signals on a few channels
- Repeatedly store and display small snapshots
- Displays signals in real time
- Measure amplitude and timing of a waveform
- Offers real-time features, like fast Fourier transform (FFT)
- Simple threshold or pulse-width triggers used to show a steady waveform

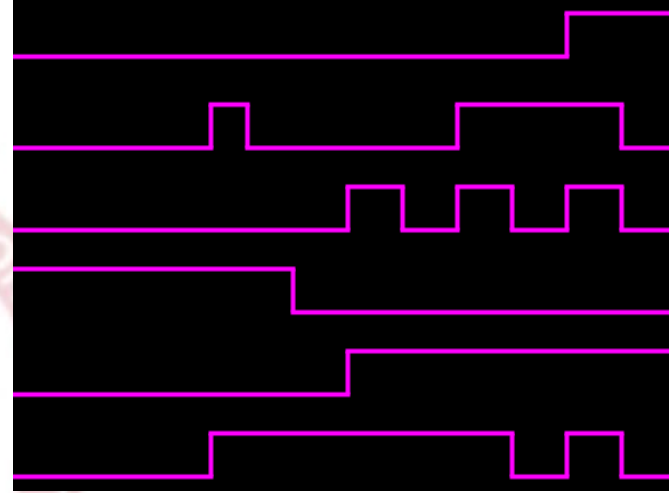
Logic Analyzer

- Measure and display digital signals on many channels
- Record data before displaying
- Allows users to navigate potentially long recordings
- Measure time between data capture points
- Offers features unique to digital systems, like protocol analyzers
- Complex triggering system used to capture and filter data

DSO and Logic analyzer

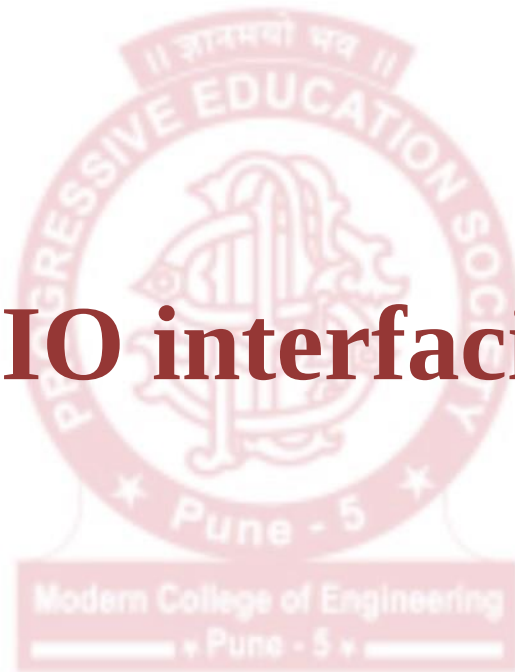


DSO output 2 to 4 channels at a time



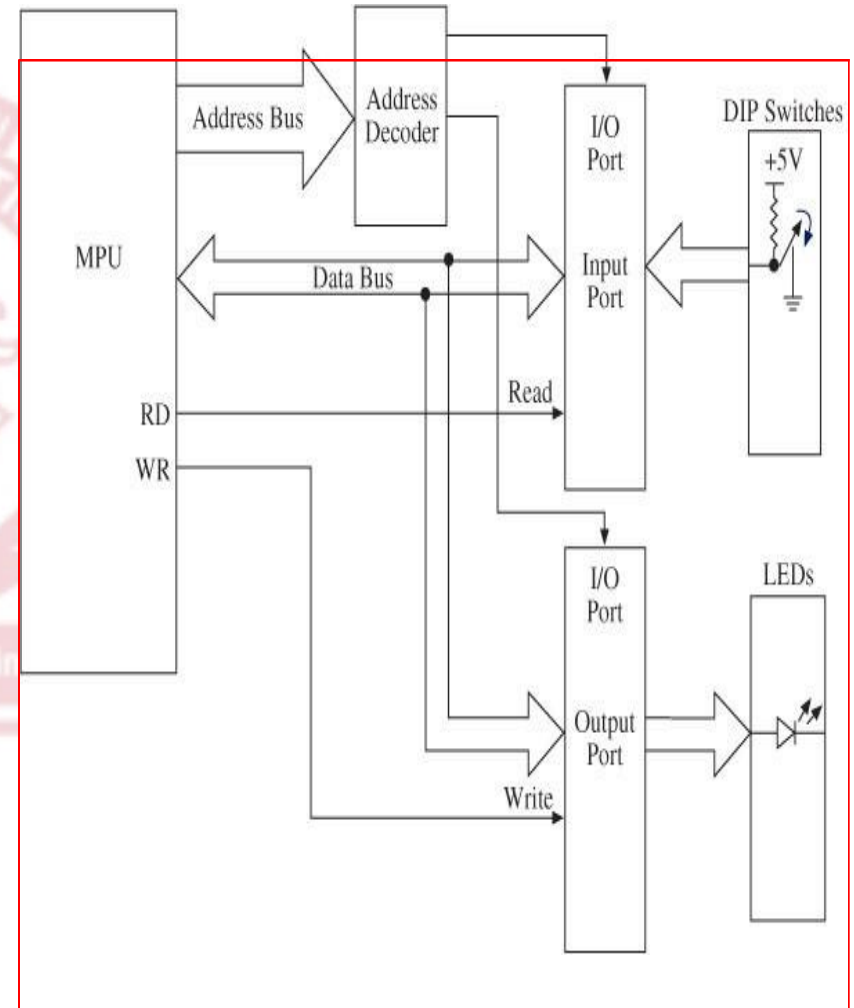
Logic analyzer output upto 6 channels at a time

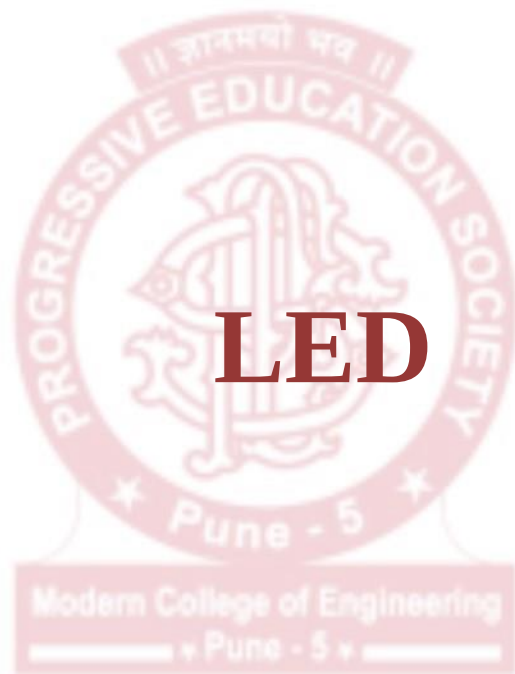
IO interfacing



Port interfacing

- To **read (receive)** binary data from an input peripheral
- MPU places the address of an input port on the **address bus**, enables the input port by asserting the **RD** signal, and reads data using the data bus.
- To **write (send)** binary data to an output peripheral
- MPU places the address of an output port on the **address bus**, places data on data bus, and asserts the **WR** signal to enable the output port.
- Remember:
- **Writing** to the port: When the MPU sends **out** or transfers data to an output port
- **Reading** from the port: When the MPU **receives(in)** data from an input port





LED

LED

Why LED

- Low Power Consumption
- Small Size
- Fast Switching
- Physically Robust
- Long Lasting
- Variety

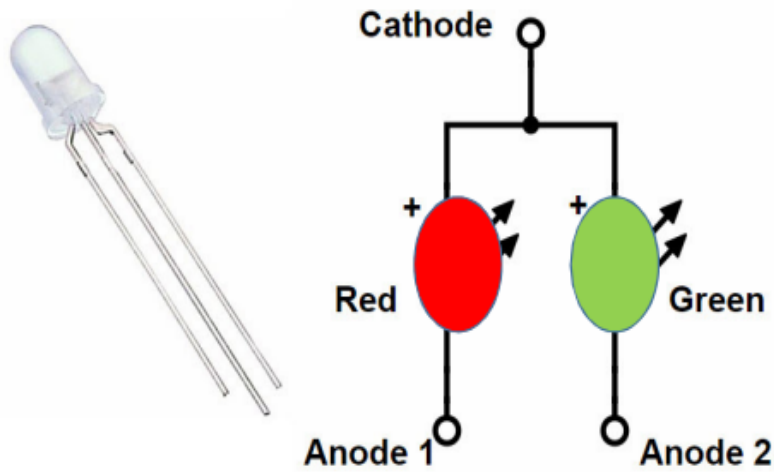
LEDs

3mm, 5mm and 8mm LEDs. These LEDs are available in different colors like Red, Blue, Yellow, Green, White, etc.



Types of LEDs

Bi-color LEDs



SMD LEDs (Surface Mount Light Emitting Diodes)

SMD LEDs are usually differentiated based on their physical dimensions. For example, the most common SMD LEDs are 3528 and 5050.

SMD 3528 LED package has a width of 3.5 mm, and a length of 2.8 mm.

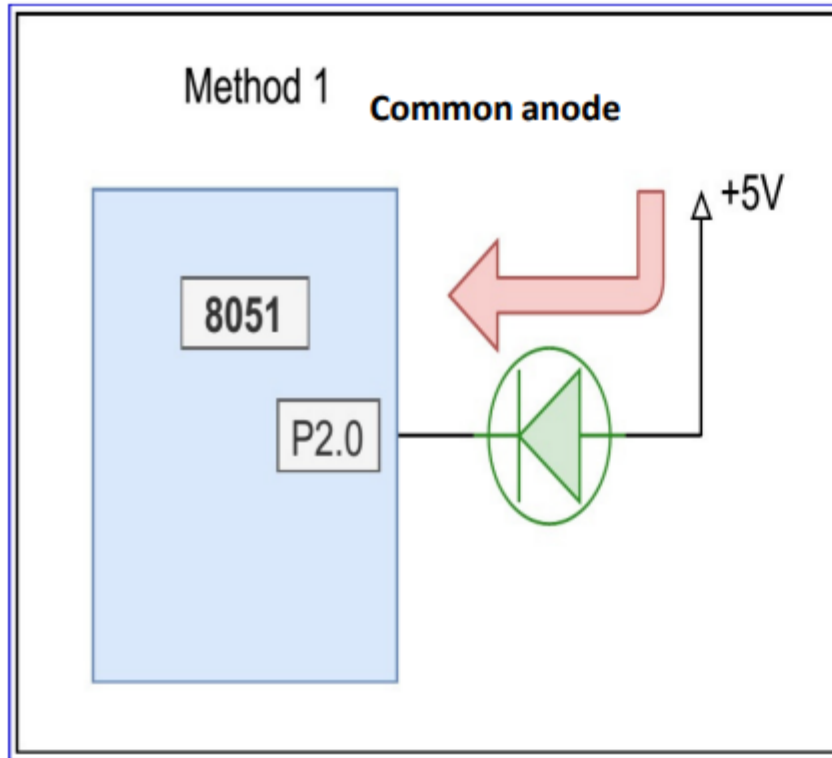
SMD 5050 :
dimensions of the chips are 5.0mm x 5.0mm.
They have 3 **LED** diodes in one housing

Tri colour LED

RGB LED (Red – Blue – Green LED)

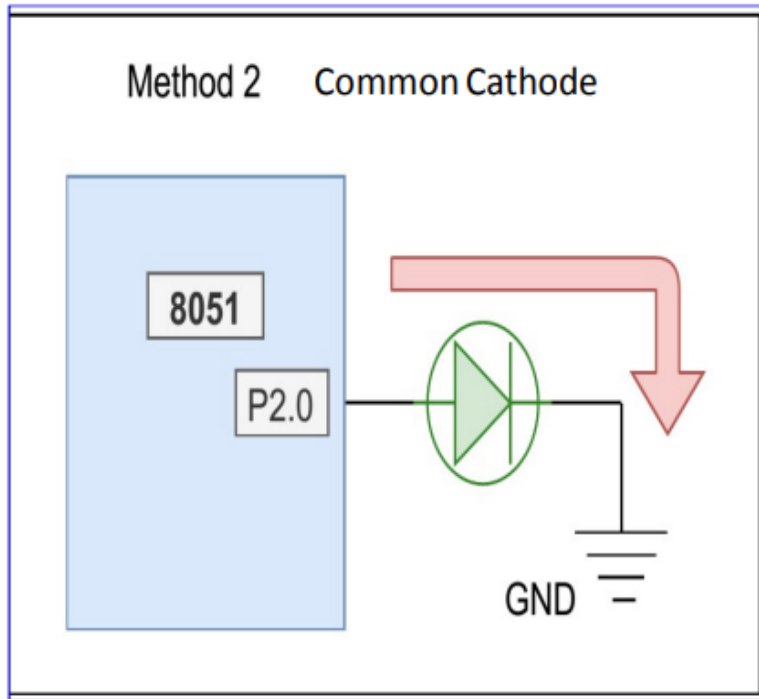


Interfacing of LED with 8051 without interrupt – Common anode



```
// Led Flashing -----  
#include<reg51.h>  
#define Del 300  
Sbit LED = P2^0;           // Defining LED Pin  
void delay(unsigned int x)  // Delay function  
{  
    unsigned int i,j;  
    for(i=0;i<x;i++)  
        for(j=0;j<=100;j++);  
}  
void main()  
{  
    while(1)                // DO it continuously  
    {  
        LED = 0;            // LED ON  
        delay(Del);  
        LED = 1;            // LED OFF  
        delay(Del);  
    }  
}
```

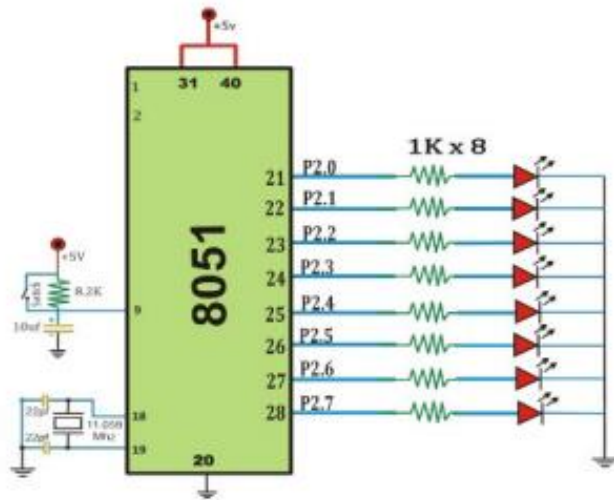
Interfacing of LED with 8051 without interrupt – Common cathode



```
// Led Flashing -----  
#include<reg51.h>  
#define Del 300  
Sbit LED = P2^0;           // Defining LED Pin  
void delay(unsigned int x)  // Delay function  
{  
    unsigned int i,j;  
    for(i=0;i<x;i++)  
        for(j=0;j<=100;j++);  
}  
void main()  
{  
    while(1)                // DO it continuously  
    {  
        LED = 1;            // LED ON  
        delay(Del);         // LED OFF  
        LED = 0;  
        delay(Del);  
    }  
}
```

Interfacing of multiple LEDs with 8051 without interrupt - Flashing

Circuit Diagram - LED Interfacing with 8051



Deepak Kumar Tiwari

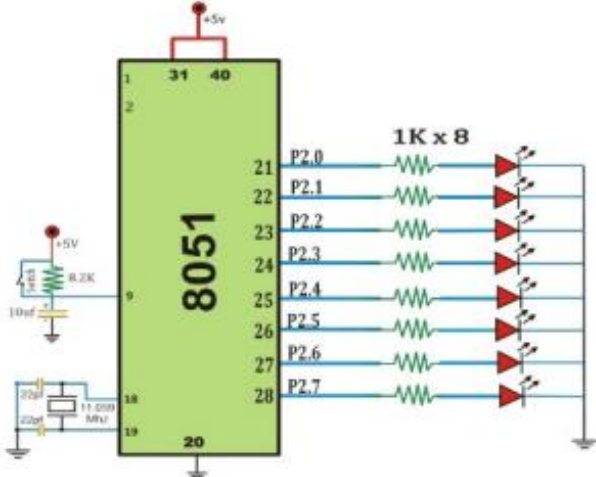
```
// Led Flashing -----  
#include<reg51.h>  
#define Del 300  
sfr led_port=0x82; // Port 2  
void delay(unsigned int x)  
{  
    unsigned int i,j;  
    for(i=0;i<x;i++)  
        for(j=0;j<=100;j++);  
}  
void main()  
{  
    while(1)  
    {  
        led_port =0xff;  
        delay(Del);  
        led_port=0x00;  
        delay(Del);  
    }  
}
```

Interfacing of multiple LEDs with 8051 without interrupt and alternate blinking program

0	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

= 55H

Circuit Diagram - LED Interfacing with 8051



Dileep Kumar Tripathi

Courtesy: Google images

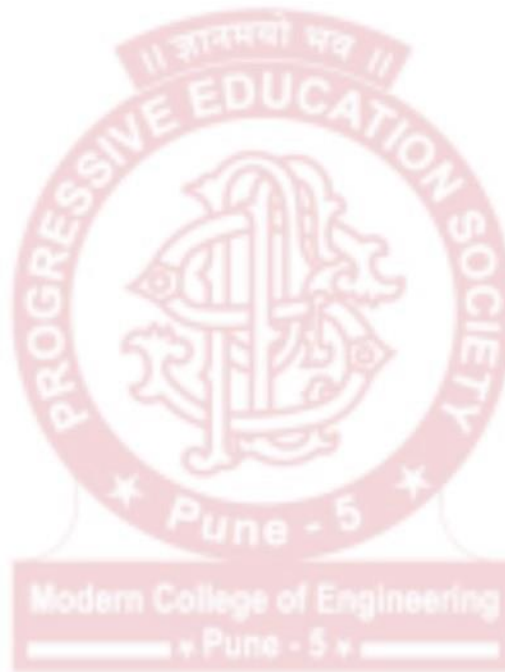
= AAH

1	0	1	0	1	0	1	0
---	---	---	---	---	---	---	---

```
// Led Flashing -----  
#include<reg51.h>  
#define Del 300  
sfr led_port=0x82; // Port 2  
void delay(unsigned int x)  
{  
    unsigned int i,j;  
    for(i=0;i<x;i++)  
        for(j=0;j<=100;j++);  
}  
void main()  
{  
    while(1)  
    {  
        led_port=0x55;  
        delay(Del);  
        led_port=0xAA;  
        delay(Del);  
    }  
}
```

Interfacing of LEDs with 8051 with Timer and alternate blinking program

```
#include<reg51.h>
void MSDelay (void)
{
    TMOD = 0x01;
    TL0 = 0x00;
    TH0 = 0X00;
    TR0 = 1;
    while (TF0 ==0);
}
void main (void)
{
    while(1)
    {
        P2 = 0x55;
        MSDelay();
        P2 = 0xAA;
        MSDelay();
    }
}
```





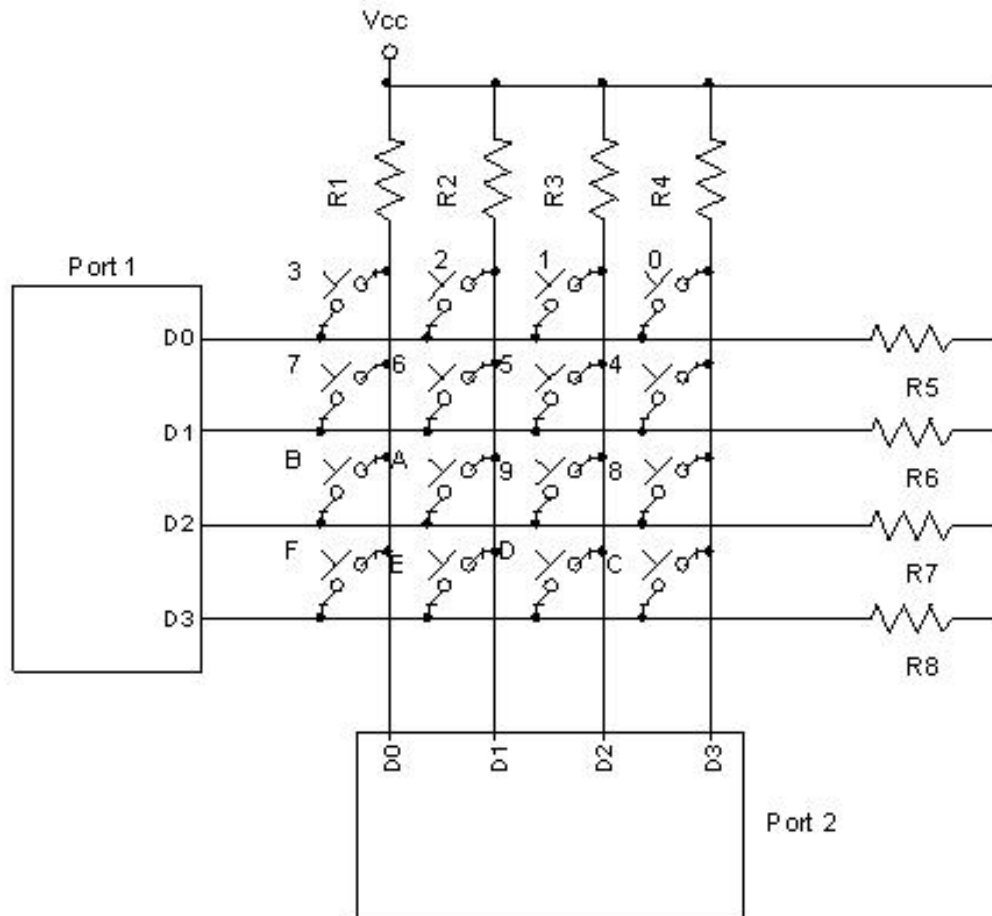
Keypad

Keypad

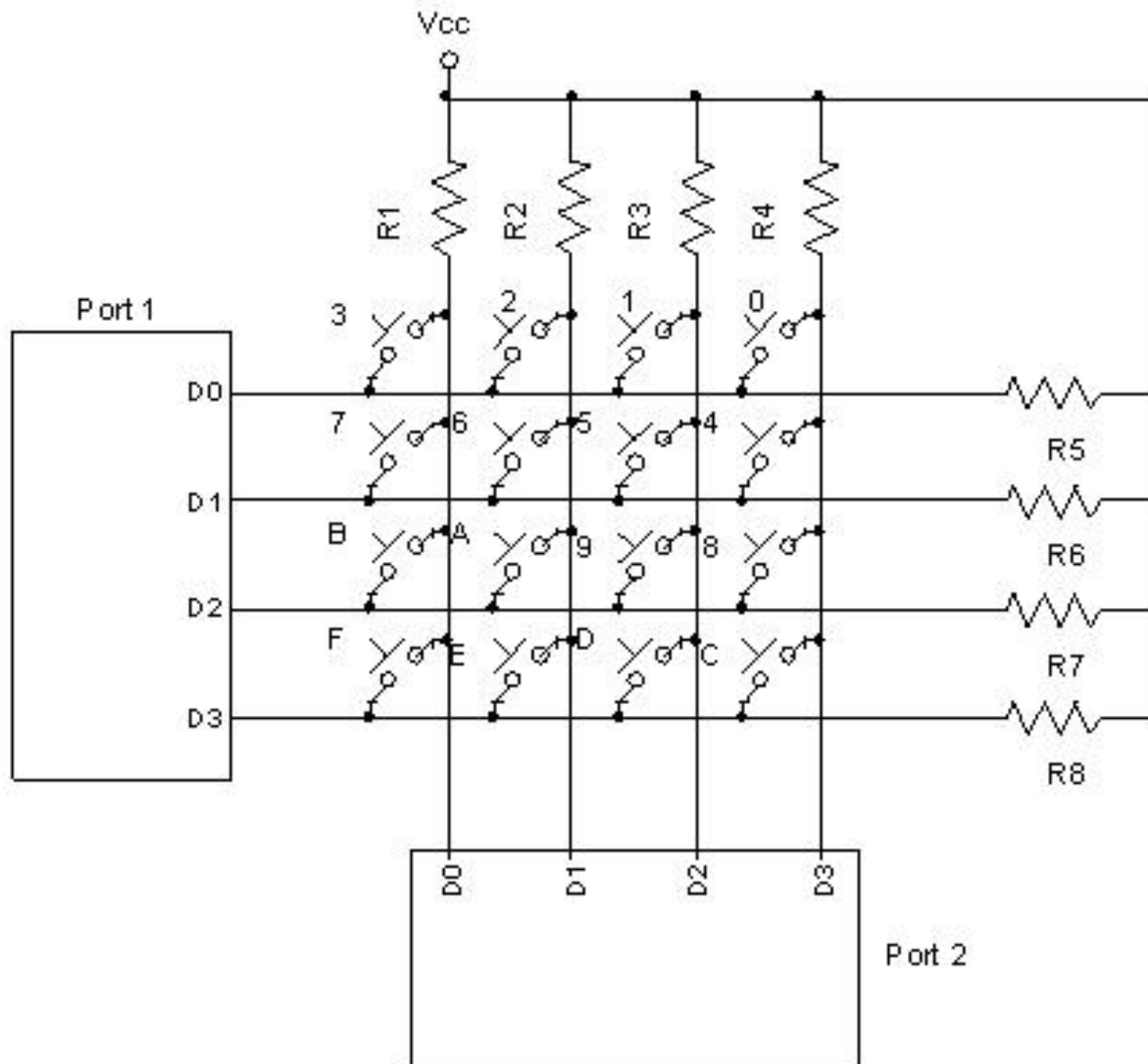
- Keypad is used as an input device to read the key pressed by user and to process it.
- 4x4 keypad consists of 4 rows and 4 columns. Switches are placed between the rows and columns. A key press establishes a connection between corresponding row and column between which the switch is placed.
- To read the key press, we need to configure the rows as outputs and columns as inputs.
- Columns are read after applying signals to the rows in order to determine whether or not a key is pressed and if pressed, which key is pressed.

Keypad

Keyboards are organized in a matrix of rows and columns. When a key is pressed, a row and a column make a contact. Otherwise, there is no connection between rows and columns.

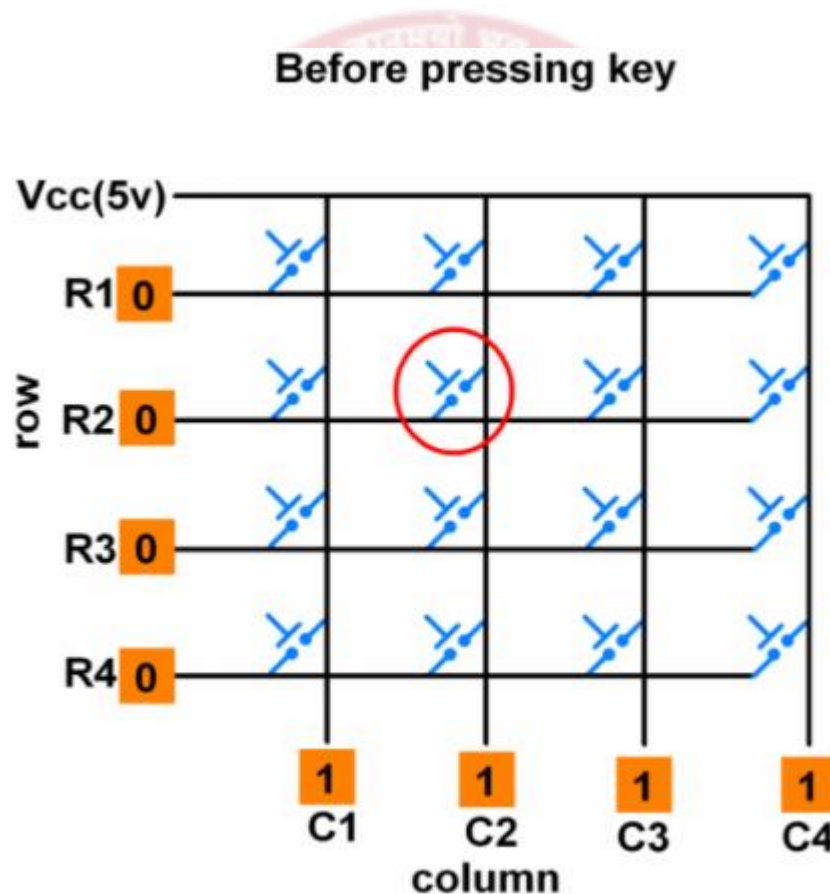


Interfacing diagram of keypad with 8051



Working of Keypad matrix

To detect a pressed key, the microcontroller grounds all rows by providing 0 to the output latch, and then it reads the columns shown in above fig.



R1 R2 R3 R4 are 0000

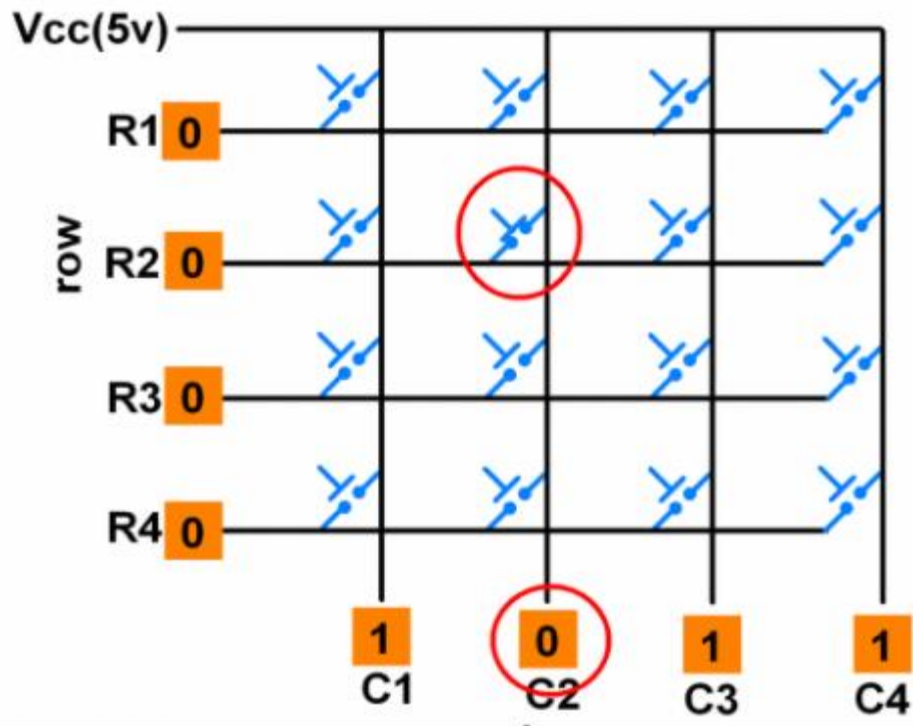
C1 C2 C3 C4 are 1111

Keypad

If the data read from columns is = 1111, no key has been pressed shown in earlier fig. and the process continues till key press is detected.

Now, consider highlighted key in above fig. is pressed. After pressing key, it makes contact of row with column shown below.

After pressing key



R1 R2 R3 R4 are 0000
C1 C2 C3 C4 are 1011

Keypad

If one of the column bits has a zero, this means that a key press has occurred. For example, if $C1:C4 = 1011$, this means that a key in the C2 column has been pressed.

After detecting a key press, microcontroller will go through the process of identifying the key.

Process of Identifying the Key

Starting from the top row, the microcontroller will ground it by providing a low to row R1 only.

Now read the columns, if the data read is all 1s, no key in that row is pressed and the process continues for the next row.

So, now ground the next row, R2. Read the columns, check for any zero and this process continues until the row is identified.

E.g. In above case we will get row 2 in which column is not equal to 1111.

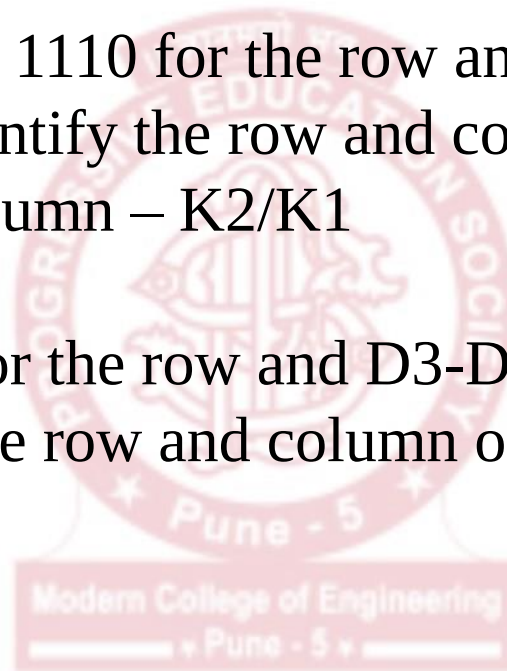
So, after identification of the row in which the key has been pressed we can easily find out the key by row and column value.

Keypad

Lets solve this first -

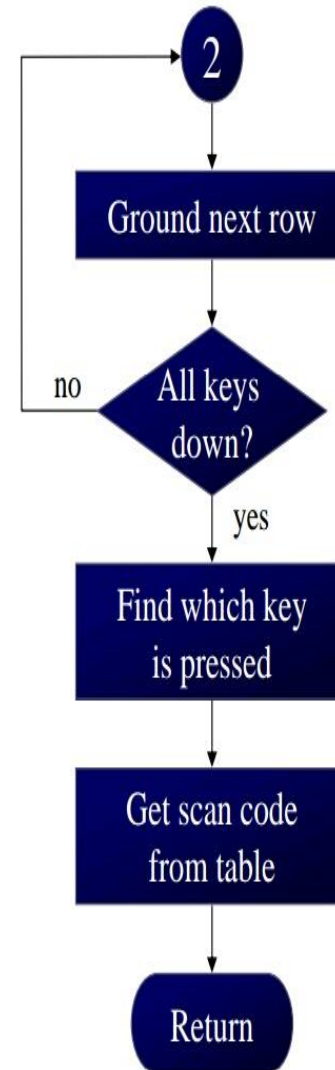
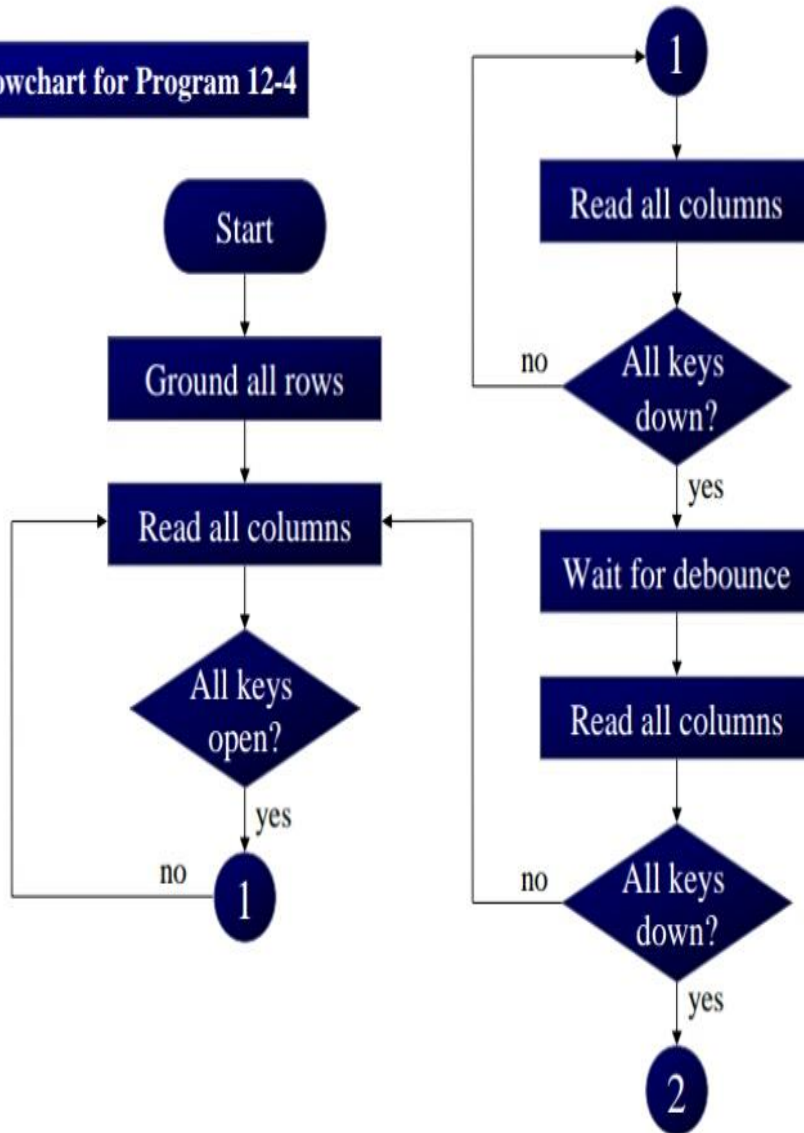
If $D3-D2-D1-D0 = 1110$ for the row and $D3-D2-D1-D0 = 1011$ for the column, identify the row and column of the pressed key.
D0 row and D2 column – K2/K1

If $D3-D0 = 1101$ for the row and $D3-D0 = 0111$ for the column, identify the row and column of the pressed key.
K7/K4



Flowchart

Flowchart for Program 12-4



Program

P1.0 TO P1.3 ARE CONNECTED TO ROWS
P2.0 TO P2.3 ARE CONNECTED TO
COLUMNS

```

    MOV P2, #0FFH
K1:  MOV P1, #00H
    MOV A,P2
    ANL A, #00001111B
    CJNE A,#00001111B, K1
K2:  ACALL DELAY
    MOV A,P2
    ANL A, #00001111B
    CJNE A,#00001111B, OVER
    SJMP K2
OVER: ACALL DELAY
    MOV A,P2
    ANL A, #00001111B
    CJNE A,#00001111B, OVER1
    SJMP K2
OVER1: MOV P1,#11111110B
    MOV A,P2
    ANL A,#00001111B
    CJNE A,#00001111B,ROW_0
    MOV P1,#11111101B
    MOV A,P2
    ANL A,#00001111B
    CJNE A,#00001111B,ROW_1
    MOV P1,#11111101B
    MOV A,P2
    ANL A,#00001111B
    CJNE A,#00001111B,ROW_2
    MOV P1,#11111101B
    MOV A,P2
    ANL A,#00001111B
    CJNE A,#00001111B,ROW_3
    LJMP K2
ROW_0: MOV DPTR,#CODE0
    SJMP FIND
ROW_1: MOV DPTR, #CODE1
    SJMP FIND
ROW_2: MOV DPTR, #CODE2
    SJMP FIND
ROW_3: MOV DPTR, #CODE3
    SJMP FIND
FIND: RRC A
    JNC MATCH
    INC DPTR
    SJMP FIND
MATCH: CLR A
    MOVC A,@A+DPTR
    MOV P0,A
    LJMP K1

    ORG 0300H

CODE0: DB '0','1','2','3' ROW 0
CODE1: DB '4','5','6','7' ROW 1
CODE2: DB '8','9','A','B' ROW 2
CODE3: DB 'C','D','E','F' ROW 3
```

Program

Write a C program to read the keypad and send the result to the first serial port.

P1.0-P1.3 connected to rows

P2.0-P1.3 connected to columns

Configure the serial port for 9600 baud, 8-bit, and 1 stop bit.

Solution:

```
#include <reg51.h>
```

```
#define COL    P2           //define ports for easier reading  
#define ROW    P1
```

```
void MSDelay(unsigned int value);  
void SerTX(unsigned char);
```

```
unsigned char keypad[4][4] =  
    { '0', '1', '2', '3',  
      '4', '5', '6', '7',  
      '8', '9', 'A', 'B',  
      'C', 'D', 'E', 'F' };
```

————— * Pune - 5 * —————

Program

```
void main()
{
    unsigned char colloc, rowloc;

    TMOD = 0x20;           //timer 1, mode 2
    TH1 = -3;              //9600 baud
    SCON = 0x50;           //8-bit, 1 stop bit
    TR1 = 1;               //start timer 1

    //keyboard routine. This sends the ASCII
    //code for pressed key to the serial port
    COL = 0xFF;             //make P2 an input port
    while(1)                //repeat forever
    {
        do
        {
            ROW = 0x00;      //ground all rows at once
            colloc = COL;     //read the columns
            colloc &= 0x0F;    //mask used bits
```

===== Pune - 5 =====

Program

```
    } while(colloc != 0x0F); //check until all keys released
do
{
    do
    {
        MSDelay(20);           //call delay
        colloc = COL;          //see if any key is pressed
        colloc &= 0x0F;         //mask unused bits
    } while(colloc == 0x0F); //keep checking for keypress

    MSDelay(20);           //call delay for debounce
    colloc = COL;          //read columns
    colloc &= 0x0F;         //mask unused bits
    } while(colloc == 0x0F); //wait for keypress

while(1)
{
    ROW = 0xFE;           //ground row 0
    colloc = COL;          //read columns
    colloc &= 0x0F;         //mask unused bits
    if(colloc != 0x0F)      //column detected
    {
```

Program

```
if(colloc != 0x0F)    //column detected
{
    rowloc = 0;      //save row location
    break;          //exit while loop
}

ROW = 0xFD;          //ground row 1
colloc = COL;        //read columns
colloc &= 0x0F;       //mask unused bits
if(colloc != 0x0F)    //column detected
{
    rowloc = 1;      //save row location
    break;          //exit while loop
}

ROW = 0xFB;          //ground row 2
colloc = COL;        //read columns
colloc &= 0x0F;       //mask unused bits
if(colloc != 0x0F)    //column detected
{
    rowloc = 2;      //save row location
    break;          //exit while loop
}
```

Program

```
        ROW = 0xF7;           //ground row 3
        colloc = COL;         //read columns
        colloc &= 0x0F;       //mask unused bits
        rowloc = 3;           //save row location
        break;                //exit while loop
    }

    //check column and send result to the serial port
    if(colloc == 0x0E)
        SerTX(keypad[rowloc][0]);
    else if(colloc == 0x0D)
        SerTX(keypad[rowloc][1]);
    else if(colloc == 0x0B)
        SerTX(keypad[rowloc][2]);
    else
        SerTX(keypad[rowloc][3]);
}

}

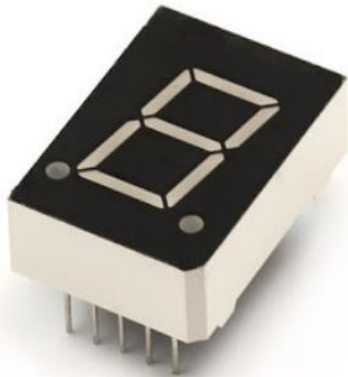
void SerTX(unsigned char x)
{
    SBUF = x;                 //place value in buffer
    while(TI==0);             //wait until transmitted
    TI = 0;                   //clear flag
}

void MSDelay(unsigned int value)
{
    unsigned int x, y;
    for(x=0;x<1275;x++)
        for(y=0;y<value;y++);
}
```

7 segment display

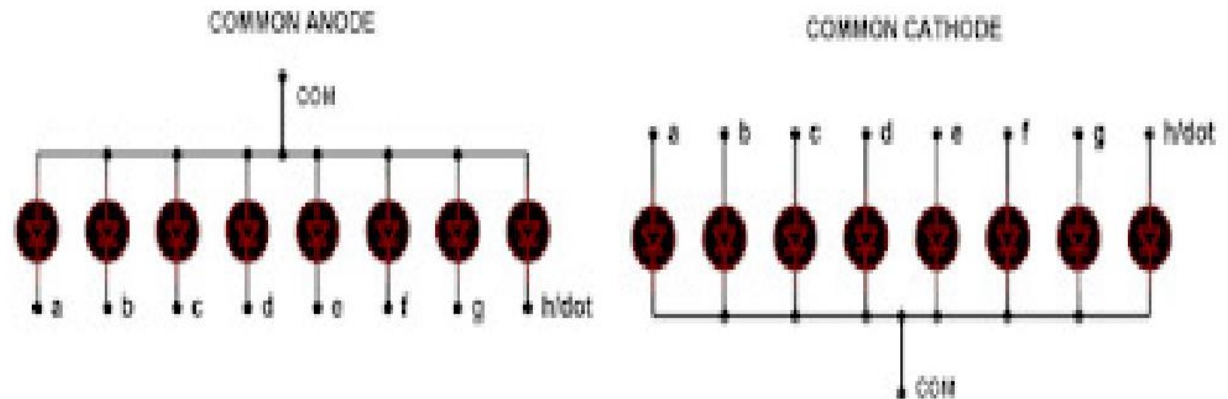


7 segment display



Common Cathode: In this all the Negative terminals (cathode) of all the 8 LEDs are connected together (see diagram below), named as COM. And all the positive terminals are left alone.

Common Anode: In this all the positive terminals (Anodes) of all the 8 LEDs are connected together, named as COM. And all the negative terminals are left alone.

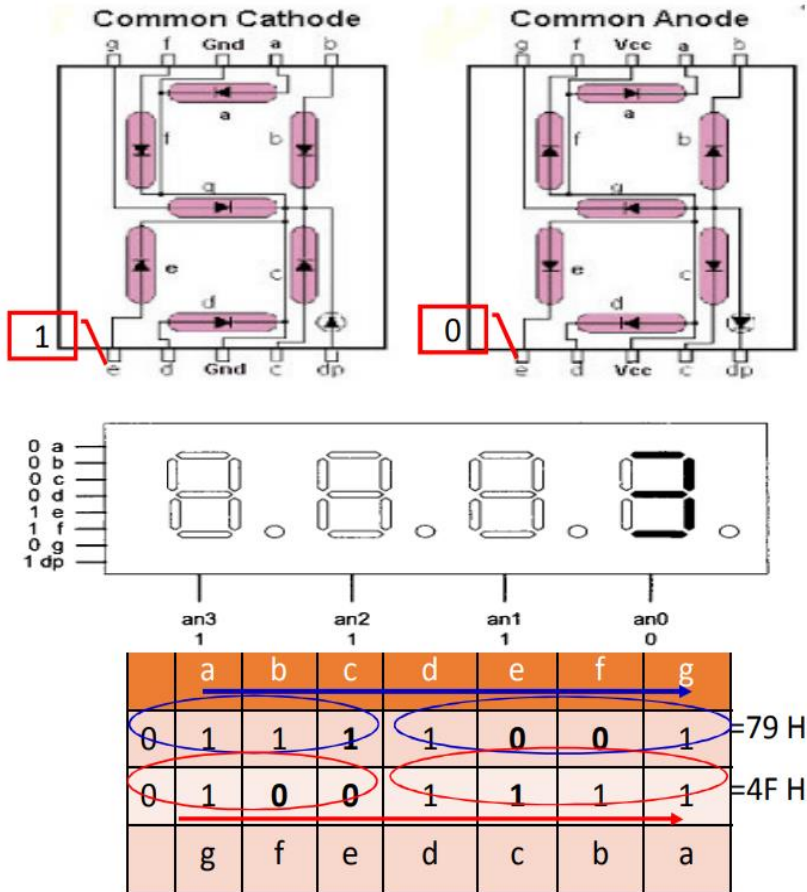


Courtesy: Google images

7 Segment display

- ❖ Can display numbers from 0 to 9 and few characters
- ❖ This display is made up of 8 LED's
- ❖ Two types
 - Common Anode
 - Anodes of all led's are tied together as common and cathodes are left alone as individual pins labeled as (a b c.....)
 - Common Cathodes
 - Cathodes of all led's are tied together as common and anodes are left alone as individual pins labeled as (a b c.....)

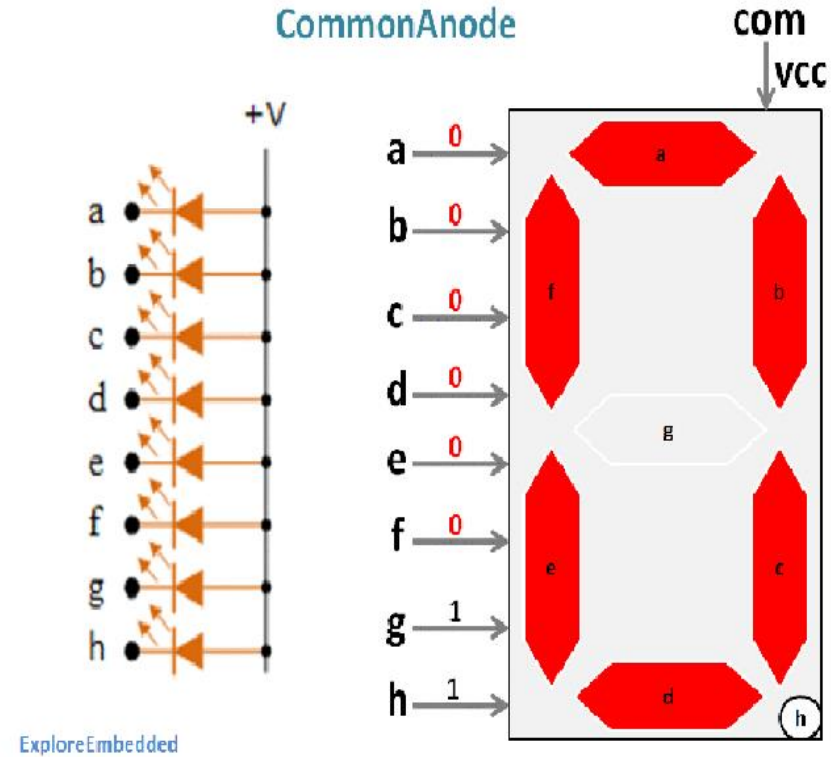
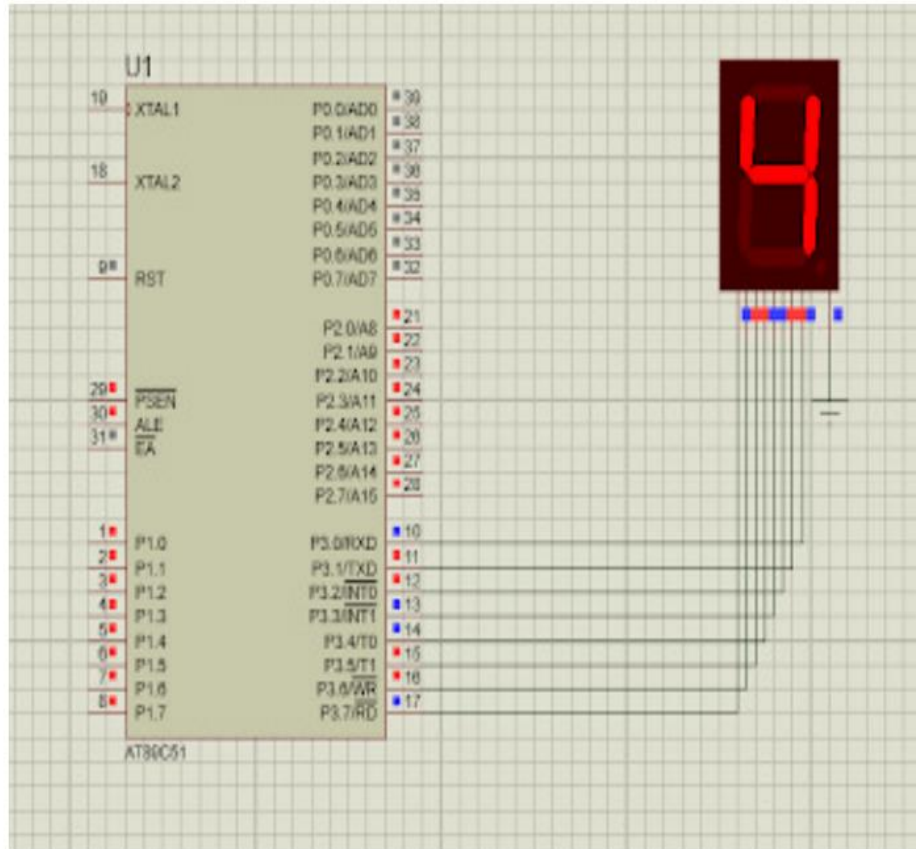
7 segment display



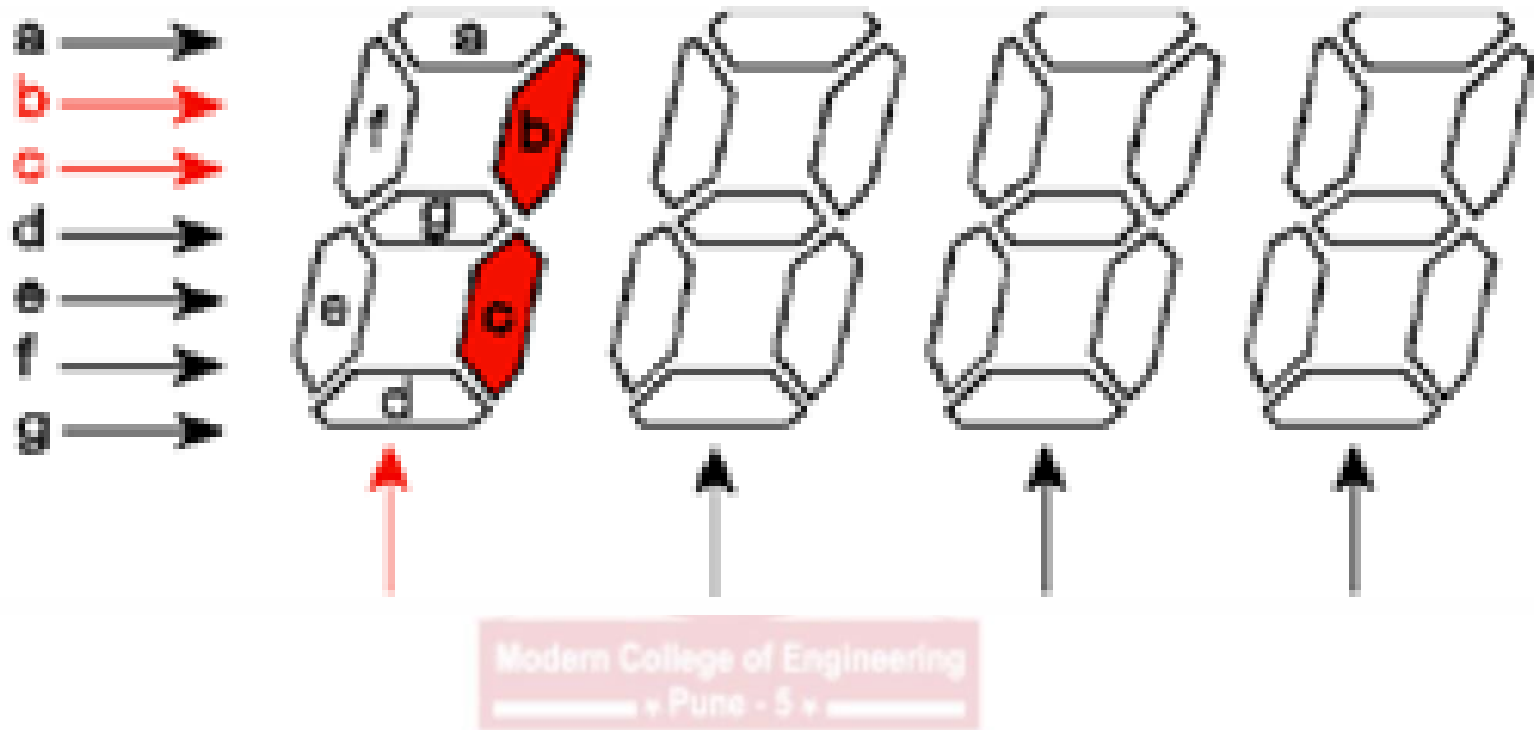
Digit	Display	gfedcba	abcdefg	a	b	c	d	e	f	g
0	0	0x3F	0x7E	on	on	on	on	on	on	off
1	1	0x06	0x30	off	on	on	off	off	off	off
2	2	0x5B	0x6D	on	on	off	on	on	off	on
3	3	0x4F	0x79	on	on	on	on	off	off	on
4	4	0x66	0x33	off	on	on	off	off	on	on
5	5	0x6D	0x5B	on	off	on	on	off	on	on
6	6	0x7D	0x5F	on	off	on	on	on	on	on
7	7	0x07	0x70	on	on	on	off	off	off	off
8	8	0x7F	0x7F	on	on	on	on	on	on	on
9	9	0x6F	0x7B	on	on	on	on	off	on	on
A	A	0x77	0x77	on	on	on	off	on	on	on
b	b	0x7C	0x1F	off	off	on	on	on	on	on
C	C	0x39	0x4E	on	off	off	on	on	on	off
d	d	0x5E	0x3D	off	on	on	on	on	off	on
E	E	0x79	0x4F	on	off	off	on	on	on	on
F	F	0x71	0x47	on	off	off	off	on	on	on

Courtesy: Google images

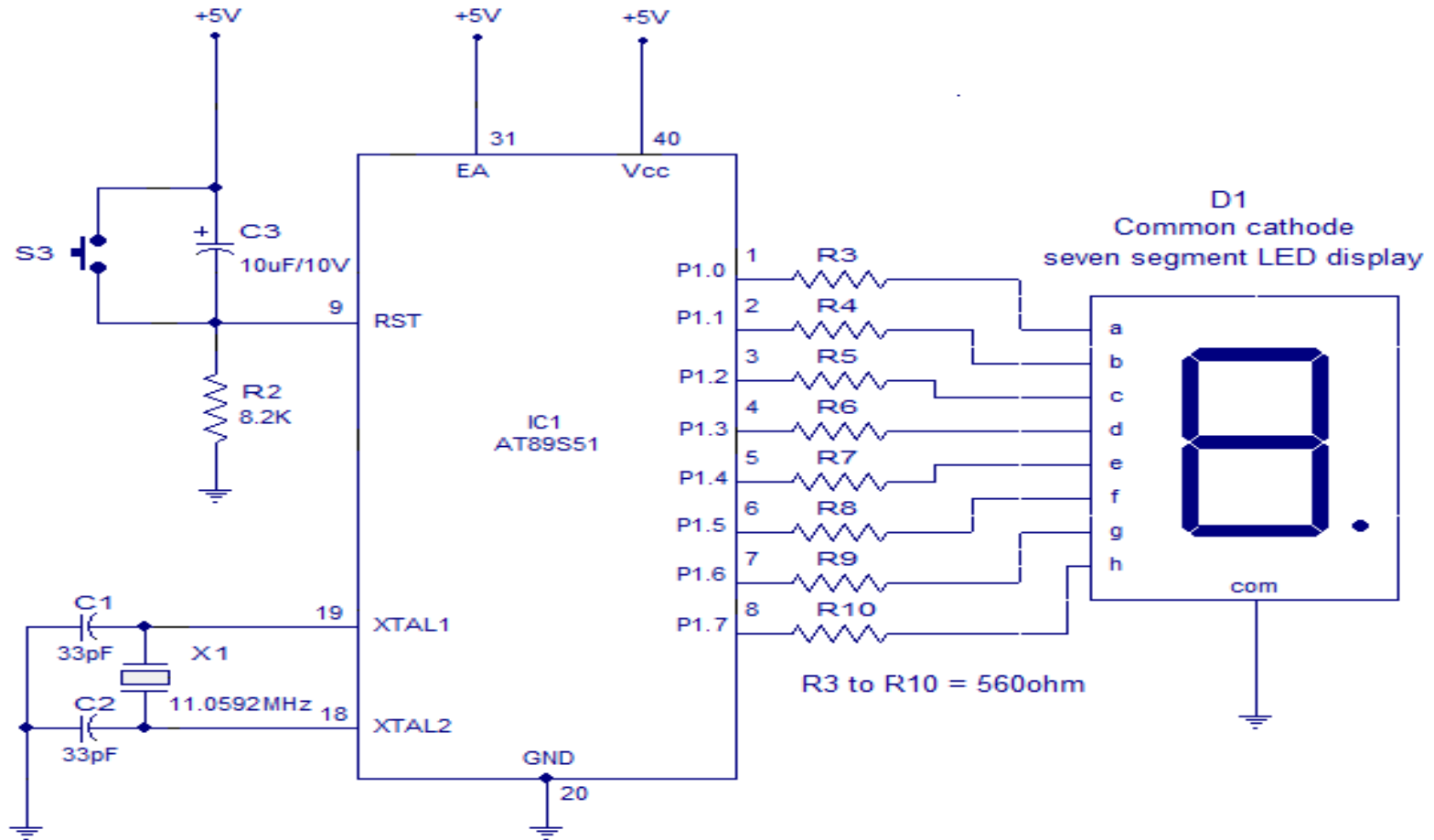
Interfacing of 7 segment display with 8051



Interfacing of 7 segment display with 8051



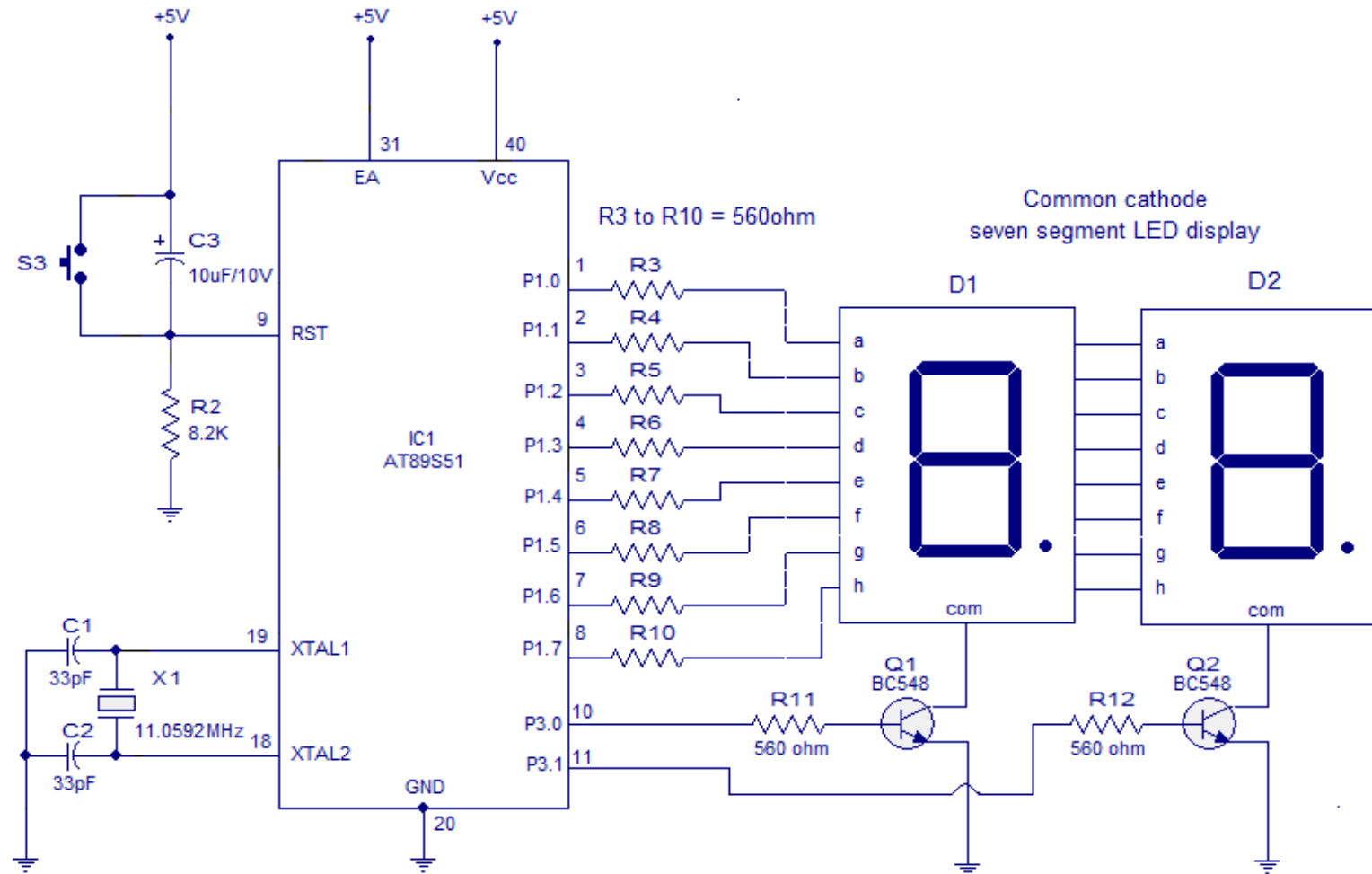
Interfacing of 7 segment display with 8051



Interfacing 7 segment display to 8051

www.circuitstoday.com

Interfacing of Multiplexed 7 segment display with 8051



Multiplexing 7 segment displays to 8051

www.circuitstoday.com

Digit drive pattern for 7 segment display

Digit	a	b	c	d	e	f	g	h (dot)	value in hex
0	1	1	1	1	1	1	0	0	3f
1	0	1	1	0	0	0	0	0	6
2	1	1	0	1	1	0	1	0	5b
3	1	1	1	1	0	0	1	0	4f
4	0	1	1	0	0	1	1	0	66
5	1	0	1	1	0	1	1	0	6d
6	1	0	1	1	1	1	1	0	7d
7	1	1	1	0	0	0	0	0	7
8	1	1	1	1	1	1	1	0	7f
9	1	1	1	1	0	1	1	0	6f

Program for interfacing 7 segment display with 8051

```
/* 7 SEGMENT DISPLAY IS CONNECTED TO P1*/
```

```
ORG 0000H
```

```
MOV P1,#00H
```

```
MOV DPTR, #DATA
```

```
UP: CLR A
```

```
ACALL DISPLAY
```

```
SJMP UP
```

```
DISPLAY: MOVC A, @A+DPTR
```

```
MOV P1.A
```

```
ACALL DELAY
```

```
INC DPTR
```

```
RET
```

```
DELAY:
```

```
MOV R0,#20H
```

```
HERE2: MOV R1,#0F0H
```

```
HERE1: DJNZ R1,HERE1
```

```
DJNZ R0,HERE2
```

```
RET
```

```
ORG 0200H
```

```
DATA:
```

```
DB 3fH
```

```
DB 06 H
```

```
DB 5b H
```

```
DB 4f H
```

```
DB 66H
```

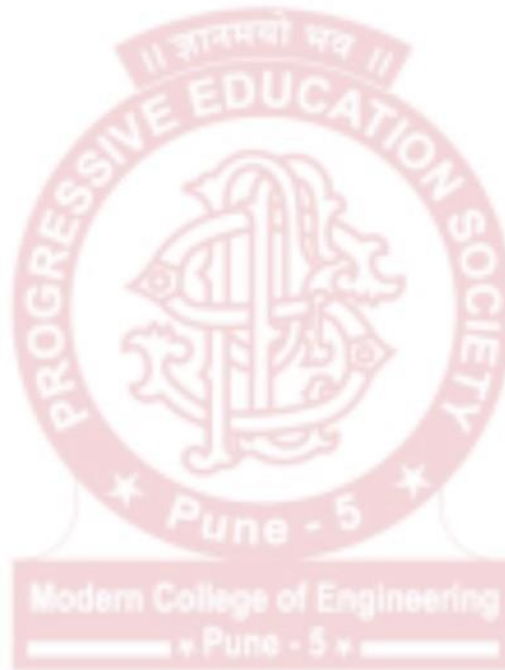
```
DB 6d H
```

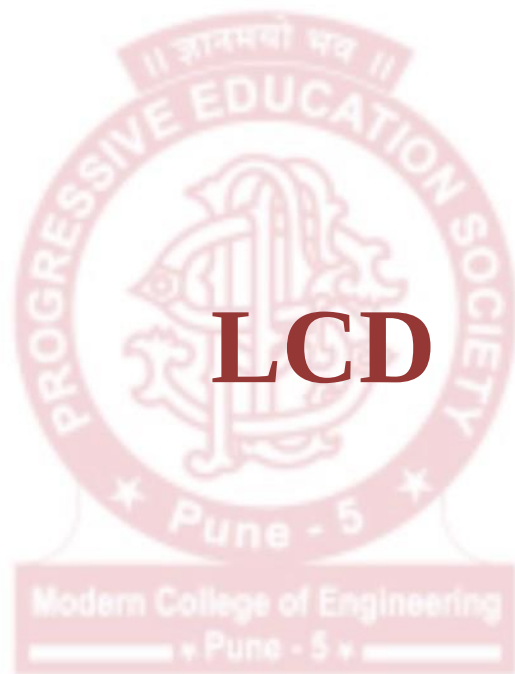
```
DB 7dH
```

```
DB 07 H
```

```
DB 7f H
```

```
DB 6f H
```

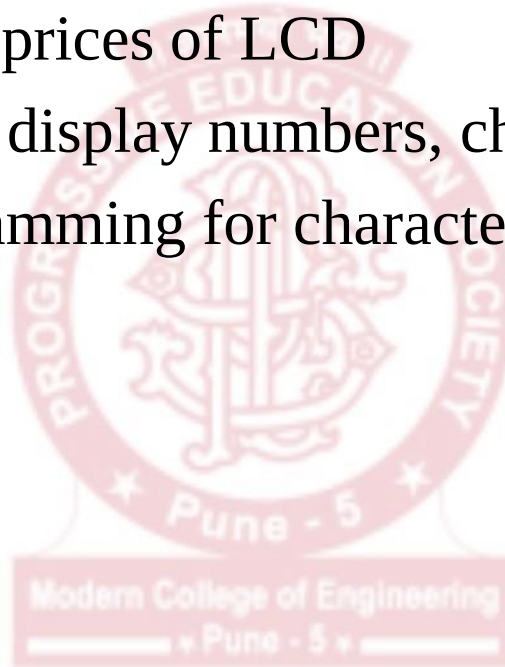




LCD

LCD Operation

- LCD is finding widespread use replacing LEDs
 - The declining prices of LCD
 - The ability to display numbers, characters, and graphics
 - Ease of programming for characters and graphics



LCD

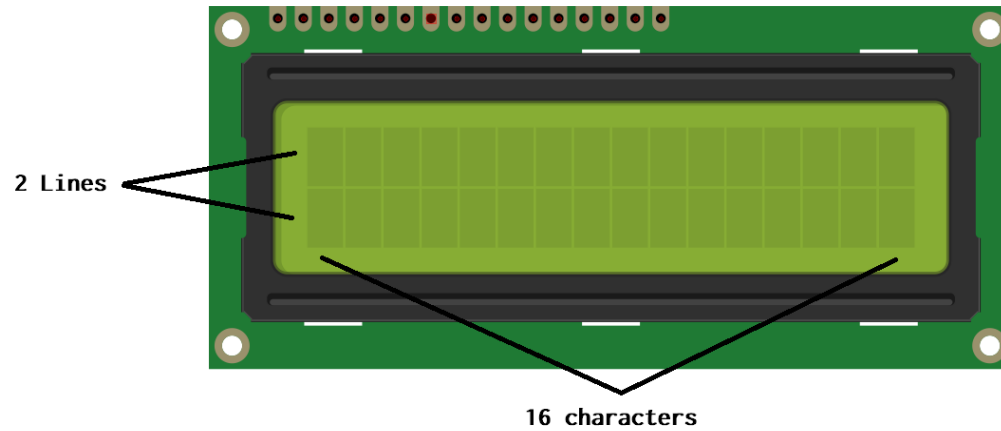


Table 12-1: Pin Descriptions for LCD

Pin	Symbol	I/O	Description
1	V _{SS}	--	Ground
2	V _{CC}	--	+5V power supply
3	V _{EE}	--	Power supply to control contrast
4	RS	I	RS = 0 to select command register, RS = 1 to select data register
5	R/W	I	R/W = 0 for write, R/W = 1 for read
6	E	I/O	Enable
7	DB0	I/O	The 8-bit data bus
8	DB1	I/O	The 8-bit data bus
9	DB2	I/O	The 8-bit data bus
10	DB3	I/O	The 8-bit data bus
11	DB4	I/O	The 8-bit data bus
12	DB5	I/O	The 8-bit data bus
13	DB6	I/O	The 8-bit data bus
14	DB7	I/O	The 8-bit data bus

- Send displayed information or instruction command codes to the LCD

- Read the contents of the LCD's internal registers

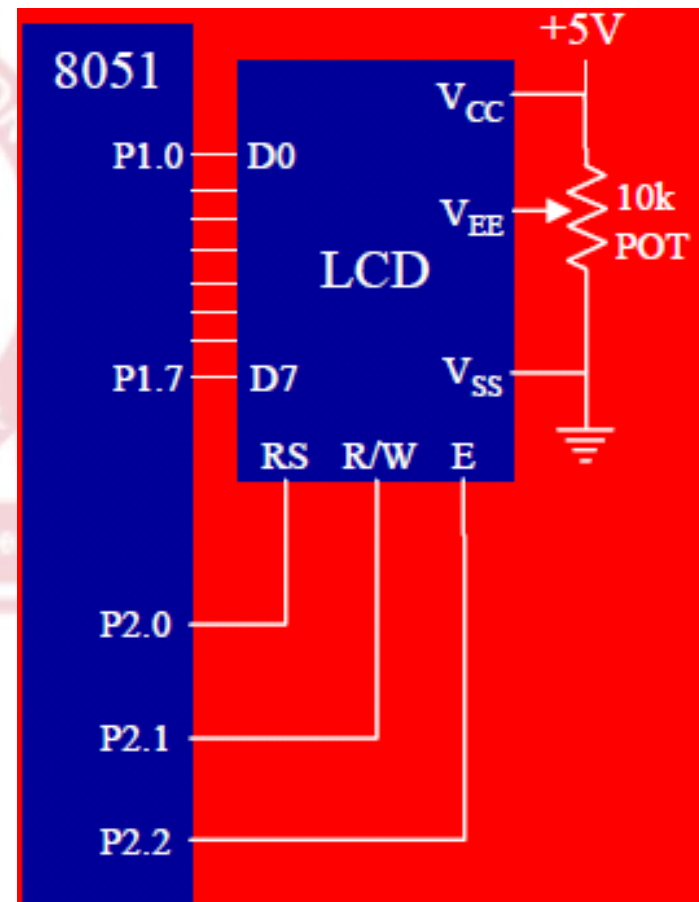
used by the LCD to latch information presented to its data bus

To send any of the commands to the LCD, make pin RS=0. For data, make RS=1. Then send a high-to-low pulse to the E pin to enable the internal latch of the LCD. This is shown in the code below.

Table 12-2: LCD Command Codes

Code	Command to LCD Instruction
(Hex)	Register
1	Clear display screen
2	Return home
4	Decrement cursor (shift cursor to left)
6	Increment cursor (shift cursor to right)
5	Shift display right
7	Shift display left
8	Display off, cursor off
A	Display off, cursor on
C	Display on, cursor off
E	Display on, cursor blinking
F	Display on, cursor blinking
10	Shift cursor position to left
14	Shift cursor position to right
18	Shift the entire display to the left
1C	Shift the entire display to the right
80	Force cursor to beginning of 1st line
C0	Force cursor to beginning of 2nd line
38	2 lines and 5x7 matrix

Note: This table is extracted from Table 12-4.



Program in C

```
#include <reg51.h>
sfr ldata = 0x90;           //P1=LCD data pins
sbit rs = P2^0;
sbit rw = P2^1;
sbit en = P2^2;
void main()
{
    lcdcmd(0x38);
    MSDelay(250);
    lcdcmd(0x0E);
    MSDelay(250);
    lcdcmd(0x01);
    MSDelay(250);
    lcdcmd(0x06);
    MSDelay(250);
    lcdcmd(0x86);           //line 1, position 6
    MSDelay(250);
    lcddata('M');
    MSDelay(250);
    lcddata('D');
    MSDelay(250);
    lcddata('E');
}
```

Program in C cont.

```
void lcdcmd(unsigned char value)
{
    ldata = value;           // put the value on t
    rs = 0;
    rw = 0;
    en = 1;                  // strobe the enable
    MSDelay(1);
    en = 0;
    return;
}
```

```
void lcddata(unsigned char value)
{
    ldata = value;           // put the value on t
    rs = 1;
    rw = 0;
    en = 1;                  // strobe the enable
    MSDelay(1);
    en = 0;
    return;
}
```

```
void MSDelay(unsigned int itime)
{
    unsigned int i, j;
    for(i=0;i<itime;i++)
        for(j=0;j<1275;j++);
}
```

Reading the Busy Flag (BF)

- There must be some delay which is needed to be there for LCD to successfully process the command or data.
- So this delay can be made either with a delay loop of specified time more than that of LCD process time or we can read the busy flag, which is recommended.
- The reason to use busy flag is that delay produced is almost for the exact amount of time for which LCD need to process the time. So is best suited for every application.
- When we send the command, the BF (busy flag) or D7th bit of the LCD becomes 1 and as soon as the command is processed the $BF = 0$.

Program to check Busy flag of LCD in C

```
#include <reg51.h>
sfr ldata = 0x90;           //P1=LCD data pins
sbit rs = P2^0;
sbit rw = P2^1;
sbit en = P2^2;
sbit busy = P1^7;
void main()
{
    lcdcmd(0x38);
    lcdcmd(0x0E);
    lcdcmd(0x01);
    lcdcmd(0x06);
    lcdcmd(0x86);           //line 1, position 6
    lcddata('M');
    lcddata('D');
    lcddata('E');
}

void lcdcmd(unsigned char value)
{
    lcdready();             //check the LCD busy flag
    ldata = value;          //put the value on the pins
    rs = 0;
    rw = 0;
    en = 1;                 //strobe the enable pin
    MSDelay(1);
    en = 0;
    return;
}
```

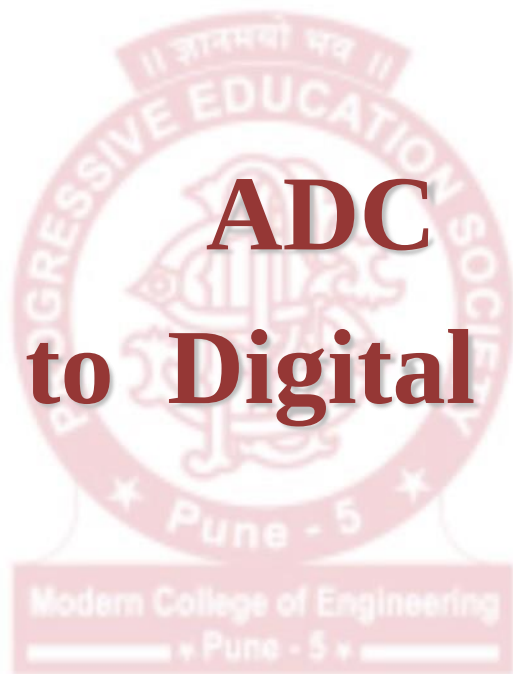
Program cont.

```
void lcddata(unsigned char value)
{
    lcdready();                //check the LCD busy flag
    ldata = value;             //put the value on the pins
    rs = 1;
    rw = 0;
    en = 1;                    //strobe the enable pin
    MSDelay(1);
    en = 0;
    return;
}

void lcdready()
{
    busy = 1;                  //make the busy pin an input
    rs = 0;
    rw = 1;
    while(busy==1)             //wait here for busy flag
    {
        en = 0;                //strobe the enable pin
        MSDelay(1);
        en = 1;
    }
    return;
}
```


ADC

(Analog to Digital Converter)



Analog and Digital signals

- **Analog signals** are continuous, with infinite values in a given range.
- **Digital signals** have discrete values such as on/off or 0/1.

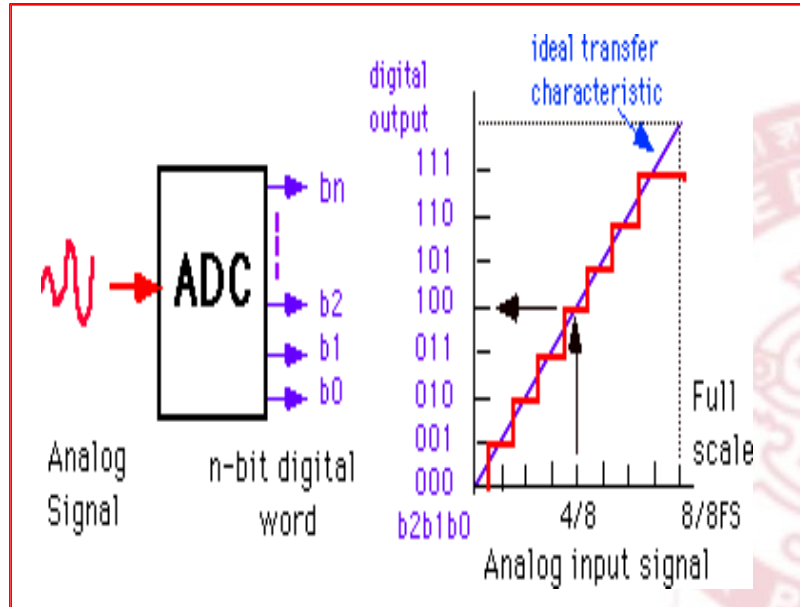
Limitations of analog signals

- Analog signals pick up noise as they are being amplified.
- Analog signals are difficult to store.
- Analog systems are more expensive in relation to digital systems.

Advantages of digital systems (signals)

- Noise can be reduced by converting analog signals in 0s and 1s.
- Binary signals of 0s/1s can be easily stored in memory.
- Technology for fabricating digital systems has become so advanced that they can be produced at low cost.
- The major limitation of a digital system is how accurately it represents the analog signals after conversion.

Typical A/D converter circuit



- A typical A/D converter circuit with one analog input signal and three digital output lines with eight possible binary combinations: 000 to 111
- The graph of digital output for corresponding analog input is as shown
- The following points can be summarized in the above process:
 - Maximum value this quantization process reaches is $7/8 V$ for a $1 V$ analog signal; includes $1/8 V$ an inherent error
 - $1/8 V$ (an inherent error) is also equal to the value of the Least Significant Bit (LSB) = 001.
 - Resolution of a converter is defined in terms of the number of discrete values it can produce; also expressed in the number of bits used for conversion or as $1/2^n$ where n = number of bits
 - The value of the most significant bit (MSB) -100- is equal to $1/2$ the voltage of the full-scale value of $1 V$.
 - The value of the largest digital number 111 is equal to full-scale value minus the value of the LSB.
 - The quantization error can be reduced or the resolution can be improved by increasing the number of bits used for the conversion

Examples

Example 1

- Assume the input analog voltage is changing between 0-5 V. Calculate the resolution. What is the maximum possible voltage out? (this is called the full-scale output) If the output is 1000 0000, what is the input?

Solution

Assumes the input analog voltage is changing between 0-5 V.

The resolution. $5 / 2^8 = 19.53 \text{ mV}$

The maximum possible voltage out – 5

If the output is 1000 0000, the input is $= \text{MaxVolt} / 2 = 2.5$

Example 2

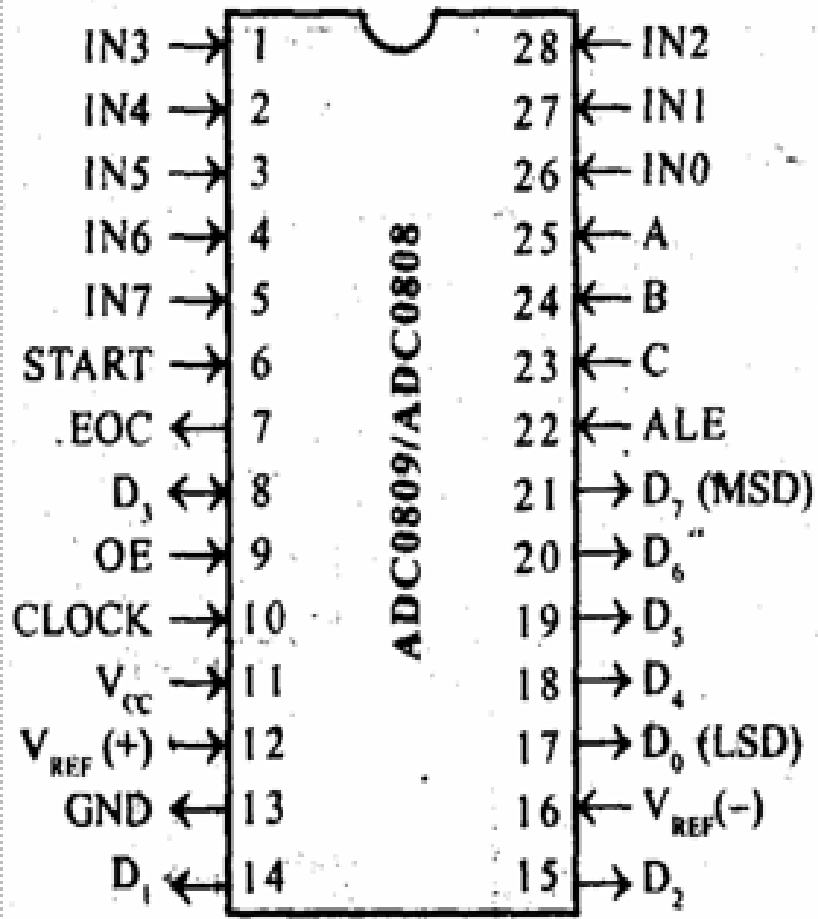
- Assumes the input analog voltage is changing between -5 to 5 V; using a 10-bit A/D converter, Calculate the number of quantization levels and the voltage resolution.

Solution

The number of quantization levels $= 2^{10}$

The voltage resolution $= 5 - (-5) / 1024 = 9.76 \text{ mV}$

ADC 0809/0808 pin out



IN0-IN7: Analog Input channels

D0-D7: Data Lines

A, B, C: Analog Channel select lines; A is LSB and C is MSB

OE: Output enable signal

ALE: Address Latch Enable

EOC: End of Conversion signal

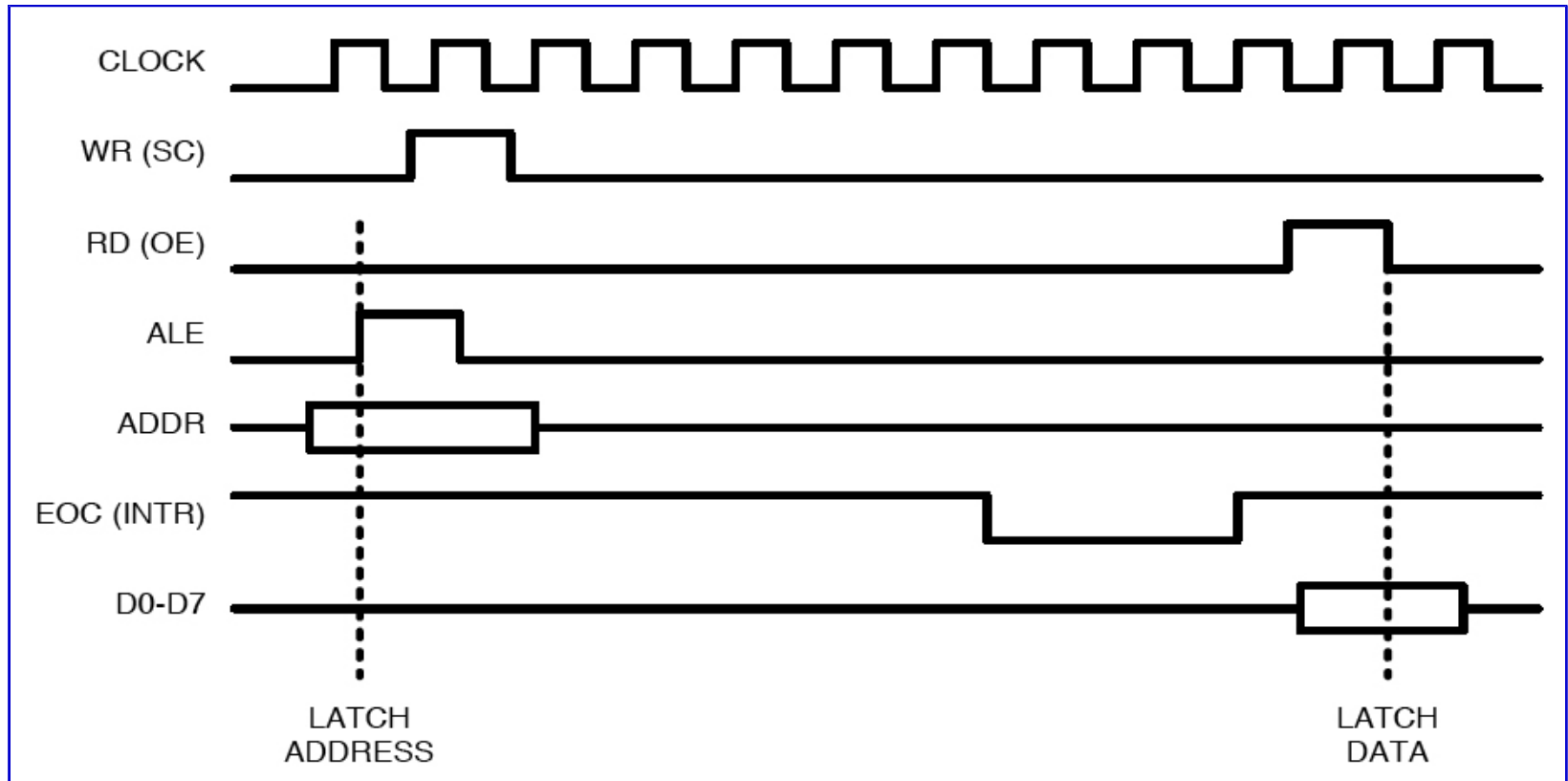
V_{ref+}/V_{ref-}: Differential Reference voltage input

Clock: External ADC clock input

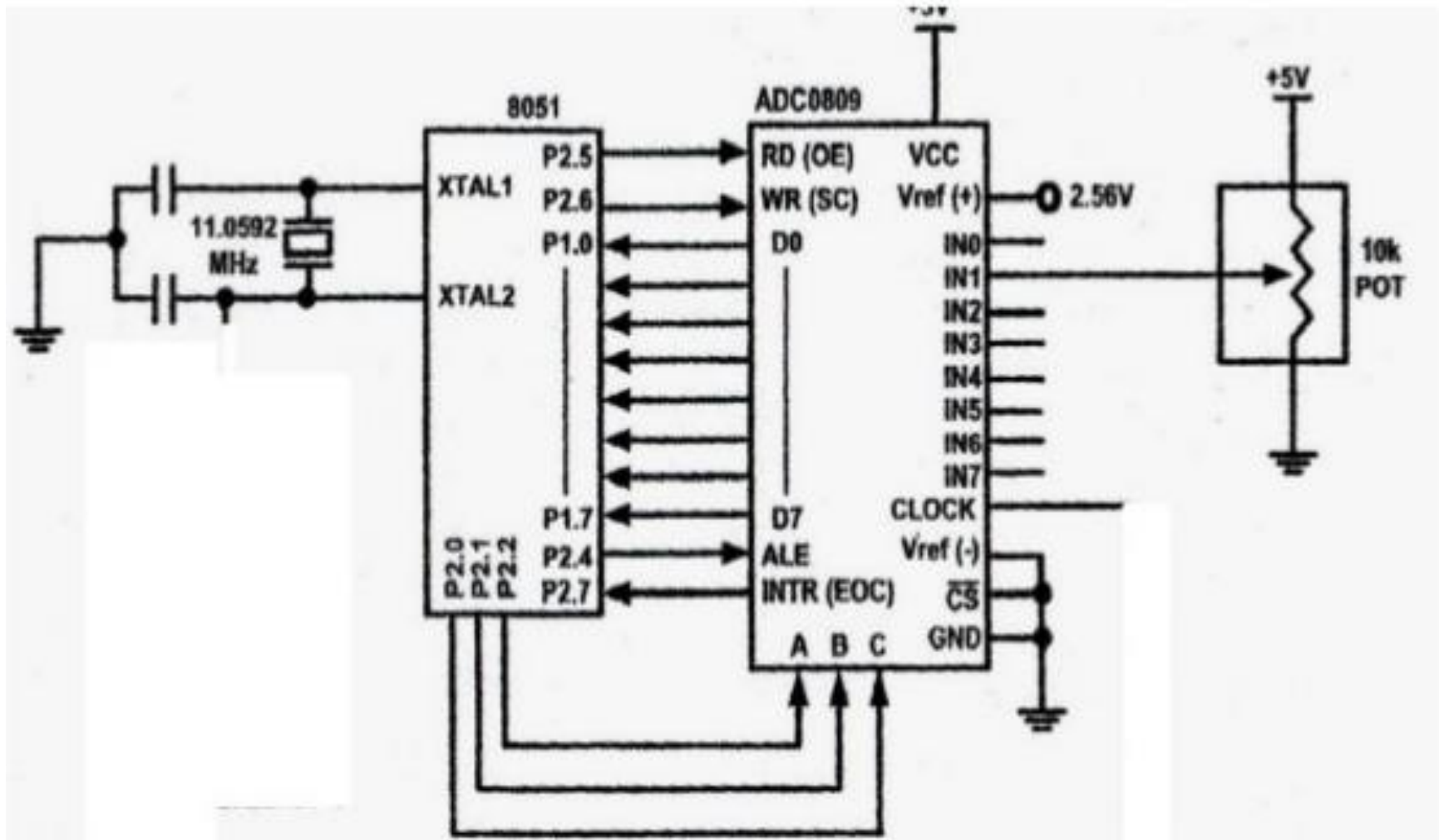
ADC

- Usually the control signals **EOC (end of conversion)**, **SC (start conversion)**, **ALE (address latch enable)** and **OE (output enable)** are interfaced by with microcontroller.
- The input control signal SC, being active-low, initiates start of conversion at falling edge of the pulse.
- The output signal EOC becomes high after completion of digitization.
- This EOC output is coupled to SC input, where falling edge of EOC output acts as SC input to direct the ADC to start the next conversion.
- As the conversion ends the EOC signal goes high and the 8-bit digital output corresponding to instantaneous value of analogue input.
- The maximum level of analogue input voltage should be appropriately scaled down below positive reference (V_{ref}) (+5V) level.

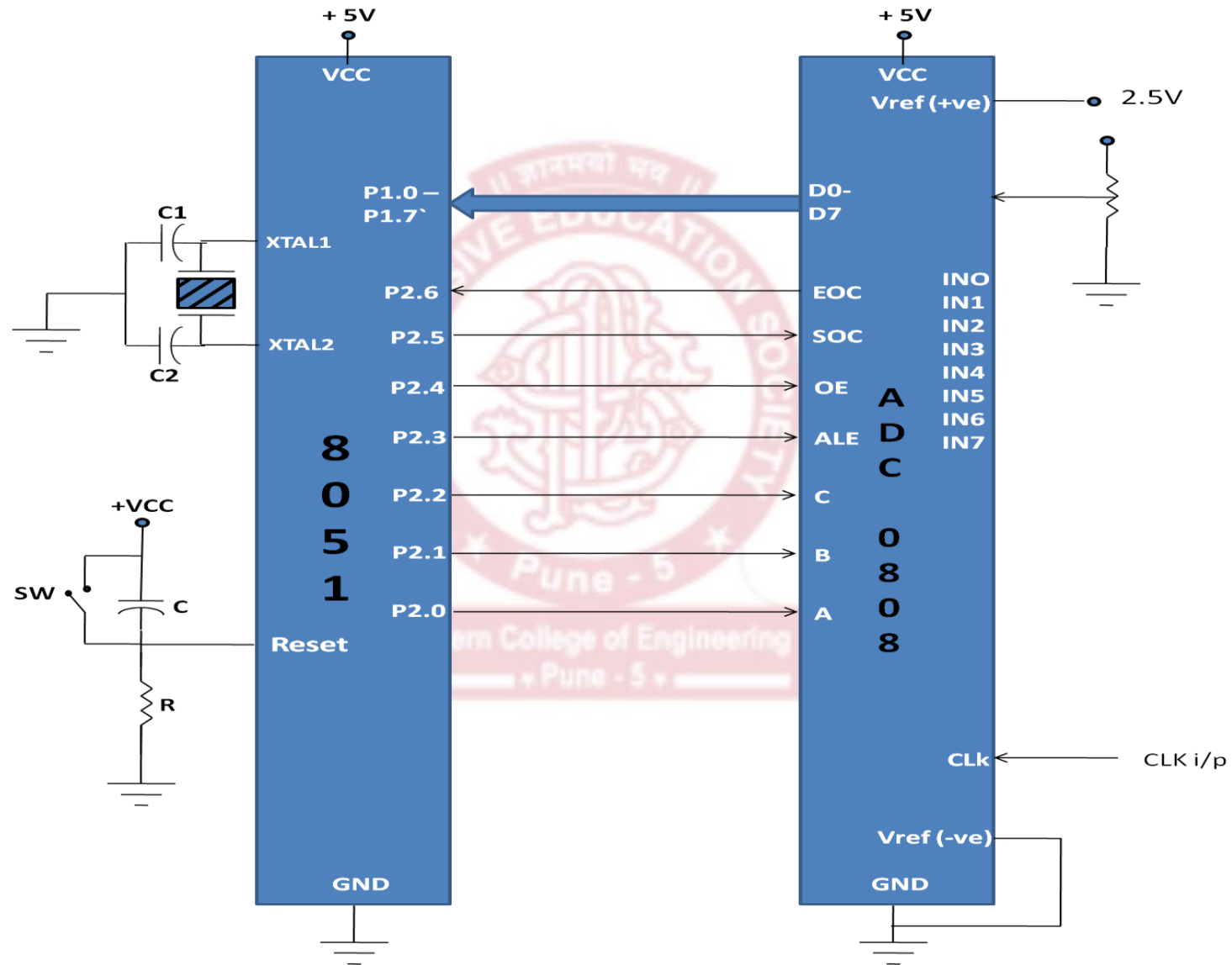
Timing Diagram



Interfacing of ADC 0809 with 8051

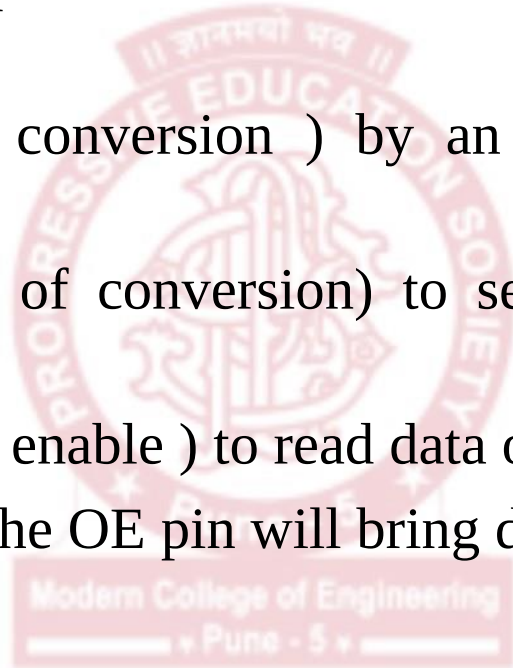


Interfacing of ADC 0808 with 8051

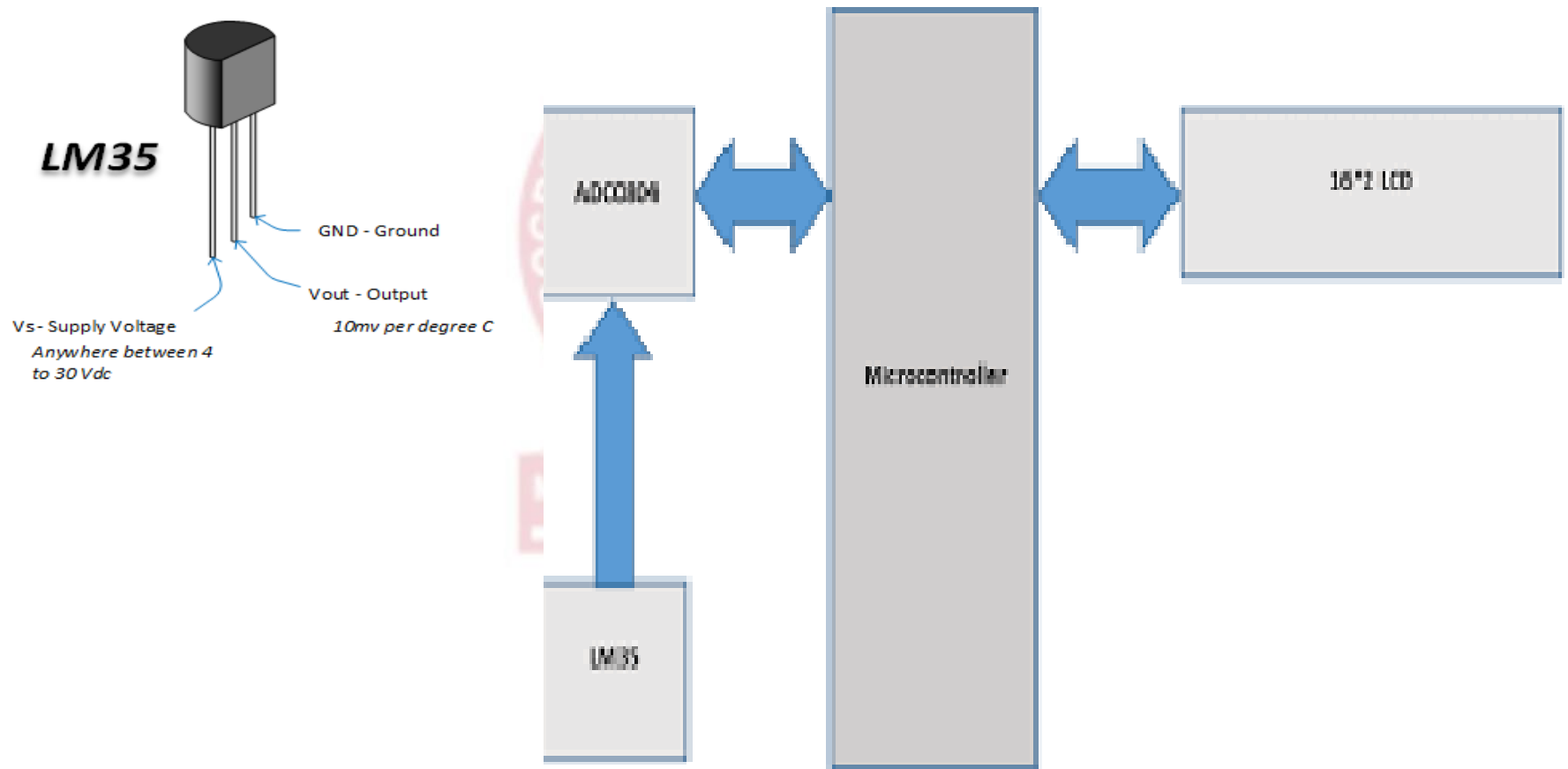


Programming steps of ADC

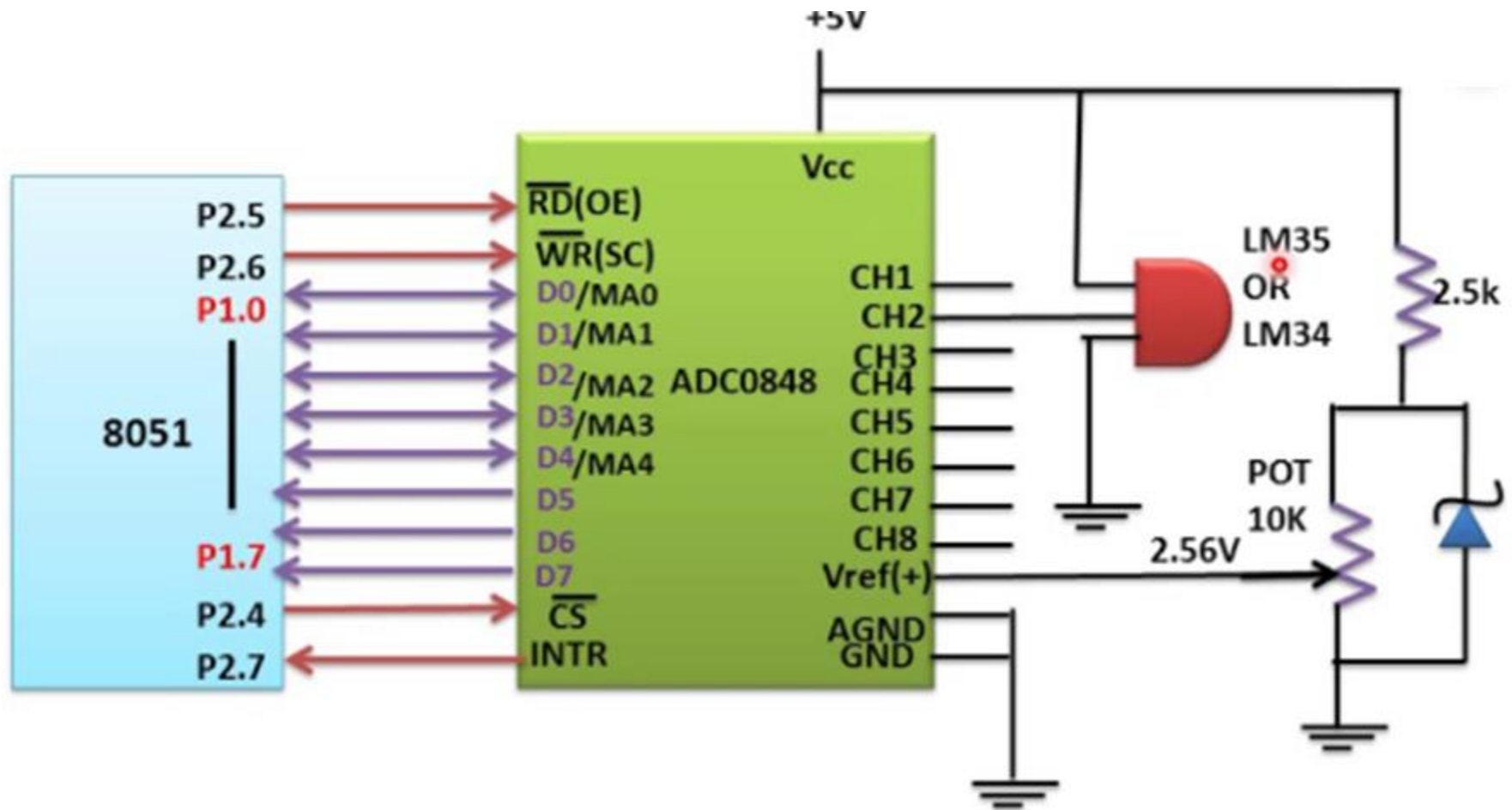
1. Select an analog channel by providing bits to A, B, and C addresses
2. Activate the ALE pin :It needs an L-to-H pulse to latch in the address
3. Activate SC (start conversion) by an H-to-L pulse to initiate conversion
4. Monitor EOC (end of conversion) to see whether conversion is finished (goes high)
5. Activate OE (output enable) to read data out of the ADC chip
6. An H-to-L pulse to the OE pin will bring digital data out of the chip



Interfacing of LM35 with 8051 using ADC



Interfacing of LM35 temperature sensor with 8051 using ADC



References:

- Textbooks prescribed in the university syllabus.
- As per the presentation slides received during Online Faculty Orientation Workshop under the aegis of BOS (E&TC), SPPU, Pune for syllabus discussions.

